Perlinpinpin

Generated by Doxygen 1.16.1

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Directory Documentation 9
5.1 src/functions Directory Reference . . . 9
5.2 src/image Directory Reference . . . 10
5.3 src/libs Directory Reference . . . 10
5.4 src/map Directory Reference . . . 11
5.5 src/noise Directory Reference . . . 11
5.6 src Directory Reference . . . 12

6 Class Documentation 13
6.1 BasicSmooth Class Reference . . . 13
6.1.1 Detailed Description . . . 14
6.1.2 Member Function Documentation . . . 14
6.1.2.1 apply() . . . 14
6.2 BiomeColor Struct Reference . . . 15
6.2.1 Detailed Description . . . 15
6.2.2 Member Data Documentation . . . 15
6.2.2.1 b . . . 15
6.2.2.2 g . . . 15
6.2.2.3 r . . . 15
6.3 ImageGenerator Class Reference . . . 15
6.3.1 Detailed Description . . . 16
6.3.2 Member Function Documentation . . . 16
6.3.2.1 generate_png_image() . . . 16
6.3.2.2 set_pixel() . . . 17
6.4 ISmoothingFunction Class Reference . . . 17
6.4.1 Detailed Description . . . 17
6.4.2 Constructor & Destructor Documentation . . . 18
6.4.2.1 ~ISmoothingFunction() . . . 18
6.4.3 Member Function Documentation . . . 18
6.4.3.1 apply() . . . 18
6.5 NoiseGenerator Class Reference . . . 18
6.5.1 Detailed Description . . . 19

6.5.2 Member Function Documentation . . . 19
6.5.2.1 getPermutationTable() . . . 19
6.5.2.2 getRandomValues() . . . 19
6.5.2.3 noise() . . . 20
6.6 PerlinGenerator Class Reference . . . 20
6.6.1 Detailed Description . . . 21
6.6.2 Constructor & Destructor Documentation . . . 22
6.6.2.1 PerlinGenerator() . . . 22
6.6.2.2 ∼PerlinGenerator() . . . 22
6.6.3 Member Function Documentation . . . 22
6.6.3.1 noise() . . . 22
6.6.4 Member Data Documentation . . . 22
6.6.4.1 GRADIENT . . . 22
6.6.4.2 permutation_table . . . 23
6.6.4.3 TRANSLATION_UNIT . . . 23
6.7 PerlinMapGenerator Class Reference . . . 23
6.7.1 Constructor & Destructor Documentation . . . 23
6.7.1.1 PerlinMapGenerator() . . . 23
6.7.2 Member Function Documentation . . . 24
6.7.2.1 generate() . . . 24
6.7.3 Member Data Documentation . . . 24
6.7.3.1 base_amplitude . . . 24
6.7.3.2 base_frequency . . . 24
6.7.3.3 height . . . 24
6.7.3.4 lacunarity . . . 24
6.7.3.5 octave . . . 24
6.7.3.6 perlin_values . . . 24
6.7.3.7 persistence . . . 24
6.7.3.8 scale . . . 24
6.7.3.9 width . . . 25
6.8 RandomGenerator Class Reference . . . 25
6.8.1 Member Function Documentation . . . 26
6.8.1.1 noise() . . . 26

7 File Documentation **27**
7.1 src/functions/BasicSmooth.h File Reference . . . 27
7.2 BasicSmooth.h . . . 28
7.3 src/functions/ISmoothingFunction.h File Reference . . . 28
7.4 ISmoothingFunction.h . . . 29
7.5 src/image/Biome.h File Reference . . . 29
7.5.1 Enumeration Type Documentation . . . 30
7.5.1.1 Biome . . . 30

7.5.2 Function Documentation . . . 30
7.5.2.1 get_biome() . . . 30
7.5.2.2 get_biome_color() . . . 31
7.6 Biome.h . . . 31
7.7 src/image/ImageGenerator.h File Reference . . . 32
7.8 ImageGenerator.h . . . 33
7.9 src/image/stb_image_write.cpp File Reference . . . 34
7.9.1 Macro Definition Documentation . . . 34
7.9.1.1 STB_IMAGE_WRITE_IMPLEMENTATION . . . 34
7.10 src/libs/stb_image_write.h File Reference . . . 35
7.10.1 Macro Definition Documentation . . . 36
7.10.1.1 STBIWDEF . . . 36
7.10.2 Typedef Documentation . . . 36
7.10.2.1 stbi_write_func . . . 36
7.10.3 Function Documentation . . . 36
7.10.3.1 stbi_flip_vertically_on_write() . . . 36
7.10.3.2 stbi_write_bmp() . . . 37
7.10.3.3 stbi_write_bmp_to_func() . . . 37
7.10.3.4 stbi_write_hdr() . . . 37
7.10.3.5 stbi_write_hdr_to_func() . . . 37
7.10.3.6 stbi_write_jpg() . . . 37
7.10.3.7 stbi_write_jpg_to_func() . . . 38
7.10.3.8 stbi_write_png() . . . 38
7.10.3.9 stbi_write_png_to_func() . . . 38
7.10.3.10 stbi_write_tga() . . . 38
7.10.3.11 stbi_write_tga_to_func() . . . 38
7.10.4 Variable Documentation . . . 39
7.10.4.1 stbi_write_force_png_filter . . . 39
7.10.4.2 stbi_write_png_compression_level . . . 39
7.10.4.3 stbi_write_tga_with_rle . . . 39
7.11 stb_image_write.h . . . 39
7.12 src/main.cpp File Reference . . . 60
7.12.1 Function Documentation . . . 61
7.12.1.1 main() . . . 61
7.13 src/map/PerlinMapGenerator.h File Reference . . . 62
7.14 PerlinMapGenerator.h . . . 62
7.15 src/noise/NoiseGenerator.h File Reference . . . 63
7.16 NoiseGenerator.h . . . 64
7.17 src/noise/PerlinGenerator.h File Reference . . . 65
7.18 PerlinGenerator.h . . . 66
7.19 src/noise/RandomGenerator.h File Reference . . . 67
7.20 RandomGenerator.h . . . 67

Index 69

Chapter 1

Directory Hierarchy

1.1 Directories

functions . . . 9
- BasicSmooth.h . . . 27
- ISmoothingFunction.h . . . 28

image . . . 10
- Biome.h . . . 29
- ImageGenerator.h . . . 32
- stb_image_write.cpp . . . 34

libs . . . 10
- stb_image_write.h . . . 35

map . . . 11
- PerlinMapGenerator.h . . . 62

noise . . . 11
- NoiseGenerator.h . . . 63
- PerlinGenerator.h . . . 65
- RandomGenerator.h . . . 67

src . . . 12
- functions . . . 9
  - BasicSmooth.h . . . 27
  - ISmoothingFunction.h . . . 28
- image . . . 10
  - Biome.h . . . 29
  - ImageGenerator.h . . . 32
  - stb_image_write.cpp . . . 34
- libs . . . 10
  - stb_image_write.h . . . 35
- map . . . 11
  - PerlinMapGenerator.h . . . 62
- noise . . . 11
  - NoiseGenerator.h . . . 63
  - PerlinGenerator.h . . . 65
  - RandomGenerator.h . . . 67
- main.cpp . . . 60

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

BiomeColor 15
ImageGenerator 15
ISmoothingFunction 17
  BasicSmooth 13
NoiseGenerator 18
  PerlinGenerator 20
  RandomGenerator 25
PerlinMapGenerator 23

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BasicSmooth
Implementation of smoothing function using polynomial values 13
BiomeColor
Biome definitions and related functions for colorization 15
ImageGenerator
Generate an image based on a Perlin Map 15
ISmoothingFunction
Interface to define a smoothing function 17
NoiseGenerator
Partially abstract class providing tools to generate a noise 18
PerlinGenerator
Implementation of a noise generator based on Perlin Algorithm 20
PerlinMapGenerator 23
RandomGenerator 25

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

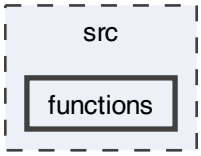
src/main.cpp 60
src/functions/BasicSmooth.h 27
src/functions/ISmoothingFunction.h 28
src/image/Biome.h 29
src/image/ImageGenerator.h 32
src/image/stb_image_write.cpp 34
src/libs/stb_image_write.h 35
src/map/PerlinMapGenerator.h 62
src/noise/NoiseGenerator.h 63
src/noise/PerlinGenerator.h 65
src/noise/RandomGenerator.h 67

Chapter 5

Directory Documentation

5.1 src/functions Directory Reference

Directory dependency graph for functions:

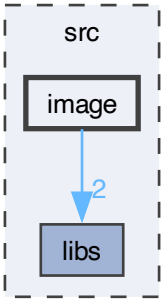


Files

- file BasicSmooth.h
- file ISmoothingFunction.h

5.2 src/image Directory Reference

Directory dependency graph for image:

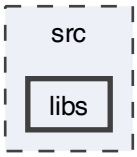


Files

- file Biome.h
- file ImageGenerator.h
- file stb_image_write.cpp

5.3 src/libs Directory Reference

Directory dependency graph for libs:

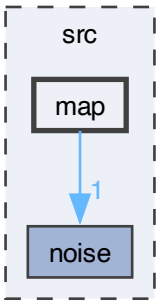


Files

- file stb_image_write.h

5.4 src/map Directory Reference

Directory dependency graph for map:

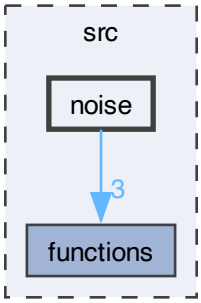


Files

- file PerlinMapGenerator.h

5.5 src/noise Directory Reference

Directory dependency graph for noise:

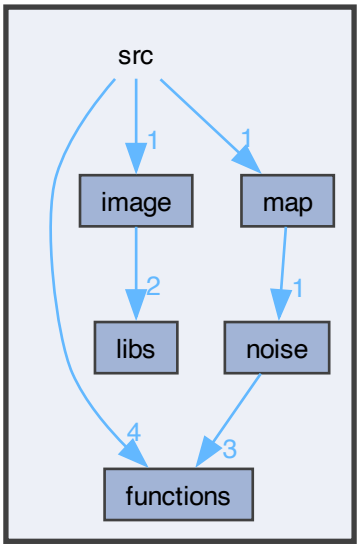


Files

- file NoiseGenerator.h
- file PerlinGenerator.h
- file RandomGenerator.h

5.6 src Directory Reference

Directory dependency graph for src:



Directories

- directory functions
- directory image
- directory libs
- directory map
- directory noise

Files

- file main.cpp

Chapter 6

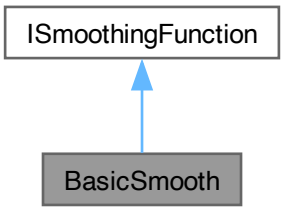
Class Documentation

6.1 BasicSmooth Class Reference

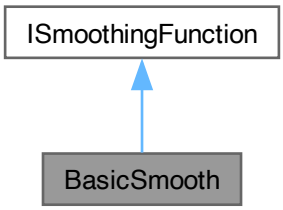
Implementation of smoothing function using polynomial values.

```
#include <BasicSmooth.h>
```

Inheritance diagram for BasicSmooth:



Collaboration diagram for BasicSmooth:



Public Member Functions

- float apply (float x) const override

  Overriding the interface with a polynamial equation such as f(x) = 6x^5-15x^4+10x^3.

Public Member Functions inherited from ISmoothingFunction

- virtual ~ISmoothingFunction ()=default

6.1.1 Detailed Description

Implementation of smoothing function using polynomial values.

See also

ISmoothingFunction

Author

Clément BARATIN

6.1.2 Member Function Documentation

6.1.2.1 apply()

```
float BasicSmooth::apply (
            float x) const  [inline], [override], [virtual]
```

Overriding the interface with a polynamial equation such as f(x) = 6x^5-15x^4+10x^3.

Parameters

x	the parameter of the function

Returns

the image of the function

Implements ISmoothingFunction.

The documentation for this class was generated from the following file:

- src/functions/BasicSmooth.h

6.2 BiomeColor Struct Reference

Biome definitions and related functions for colorization.

```
#include <Biome.h>
```

Public Attributes

- unsigned char r
- unsigned char g
- unsigned char b

6.2.1 Detailed Description

Biome definitions and related functions for colorization.

Author

Clément BARATIN

6.2.2 Member Data Documentation

6.2.2.1 b

```
unsigned char BiomeColor::b
```

6.2.2.2 g

```
unsigned char BiomeColor::g
```

6.2.2.3 r

```
unsigned char BiomeColor::r
```

The documentation for this struct was generated from the following file:

- src/image/Biome.h

6.3 ImageGenerator Class Reference

Generate an image based on a Perlin Map.

```
#include <ImageGenerator.h>
```

Static Public Member Functions

- static void generate_png_image (std::vector< std::vector< float > > perlin_value, int width, int height, const char *filename, bool colorize=false)

  Generate a png image based on a perlin noise.
- static void set_pixel (std::vector< unsigned char > &pixels, int width, int x, int y, unsigned char r, unsigned char g, unsigned char b)

6.3.1 Detailed Description

Generate an image based on a Perlin Map.

See also

PerlinGenerator

Author

Clément BARATIN

6.3.2 Member Function Documentation

6.3.2.1 generate_png_image()

```
void ImageGenerator::generate_png_image (
            std::vector< std::vector< float > > perlin_value,
            int width,
            int height,
            const char * filename,
            bool colorize = false)  [inline], [static]
```

Generate a png image based on a perlin noise.

Parameters

perlin_value	the perlin noise array.
width	the width of the image.
height	the height of the image.
filename	the filename of the png image.
colorize	whether to colorize the image based on the perlin value (biome-like) or to use a grayscale representation.

6.3.2.2 set_pixel()

```
void ImageGenerator::set_pixel (
            std::vector< unsigned char > & pixels,
            int width,
            int x,
            int y,
            unsigned char r,
            unsigned char g,
            unsigned char b)  [inline], [static]
```

The documentation for this class was generated from the following file:

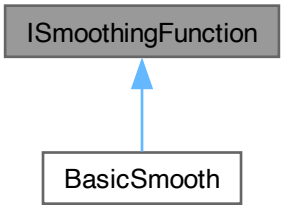
- src/image/ImageGenerator.h

6.4 ISmoothingFunction Class Reference

Interface to define a smoothing function.

```
#include <ISmoothingFunction.h>
```

Inheritance diagram for ISmoothingFunction:



Public Member Functions

- virtual ~ISmoothingFunction ()=default
- virtual float apply (float x) const =0

  Apply the smoothing function to a given value.

6.4.1 Detailed Description

Interface to define a smoothing function.

Author

Clément BARATIN

6.4.2 Constructor & Destructor Documentation

6.4.2.1 ~ISmoothingFunction()

```
virtual ISmoothingFunction::~ISmoothingFunction () [virtual], [default]
```

6.4.3 Member Function Documentation

6.4.3.1 apply()

```
virtual float ISmoothingFunction::apply (
            float x) const [pure virtual]
```

Apply the smoothing function to a given value.

Parameters

x	The input value to be smoothed.

Returns

The smoothed output value.

Implemented in BasicSmooth.

The documentation for this class was generated from the following file:

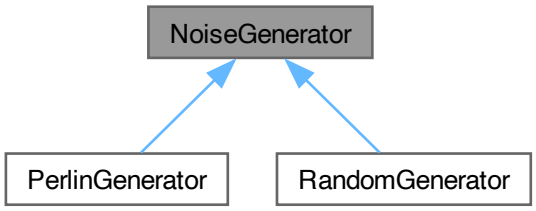
- src/functions/ISmoothingFunction.h

6.5 NoiseGenerator Class Reference

Partially abstract class providing tools to generate a noise.

```
#include <NoiseGenerator.h>
```

Inheritance diagram for NoiseGenerator:



Public Member Functions

- virtual float noise (float x, float y, ISmoothingFunction const &smoothing_function)=0

  Generate a noise for a given points. Abstract to make it modulable for differents noise generator.

Static Protected Member Functions

- static int * getPermutationTable ()

  Return a permutation table (lenght: 512).

Static Private Member Functions

- static int * getRandomValues ()

  Generate an array of 256 integers for 1 to 256 randomly ordered. Can be used to generate the permutation table for the Perlin Noise. Statically declared.

6.5.1 Detailed Description

Partially abstract class providing tools to generate a noise.

See also

PerlinGenerator

Author

Clément BARATIN

6.5.2 Member Function Documentation

6.5.2.1 getPermutationTable()

```
int * NoiseGenerator::getPermutationTable () [inline], [static], [protected]
```

Return a permutation table (lenght: 512).

Returns

int* The permutation table.

6.5.2.2 getRandomValues()

```
int * NoiseGenerator::getRandomValues () [inline], [static], [private]
```

Generate an array of 256 integers for 1 to 256 randomly ordered. Can be used to generate the permutation table for the Perlin Noise. Statically declared.

Returns

A pointer to the array.

6.5.2.3 noise()

```
virtual float NoiseGenerator::noise (
            float x,
            float y,
            ISmoothingFunction const & smoothing_function)  [pure virtual]
```

Generate a noise for a given points. Abstract to make it modulable for differents noise generator.

Parameters

x	the X value of the point.
y	the Y value of the point.
smoothing_function	a smoothing function used to make the noise more regular.

Returns

The noise for a given point.

Implemented in PerlinGenerator, and RandomGenerator.

The documentation for this class was generated from the following file:

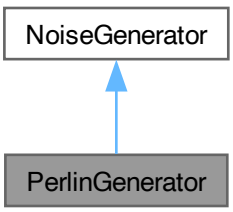
- src/noise/NoiseGenerator.h

6.6 PerlinGenerator Class Reference

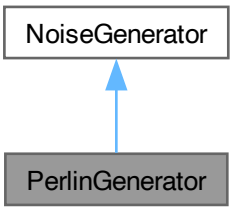
Implementation of a noise generator based on Perlin Algorithm.

```
#include <PerlinGenerator.h>
```

Inheritance diagram for PerlinGenerator:



Collaboration diagram for PerlinGenerator:



Public Member Functions

- PerlinGenerator ()

  Constructor for the PerlinGenerator. It initializes the permutation table.
- ~PerlinGenerator ()

  Destructor for the PerlinGenerator. It deletes the permutation table.
- float noise (float x, float y, ISmoothingFunction const &smoothing_function) override

  Generator for a given point a Perlin Noise.

Public Attributes

- int * permutation_table
- const float TRANSLATION_UNIT = 1.0f / std::sqrt(2.0f)
- const float GRADIENT [8][2]

Additional Inherited Members

Static Protected Member Functions inherited from NoiseGenerator

- static int * getPermutationTable ()

  Return a permutation table (lenght: 512).

6.6.1 Detailed Description

Implementation of a noise generator based on Perlin Algorithm.

See also

NoiseGenerator

Author

Clément BARATIN

6.6.2 Constructor & Destructor Documentation

6.6.2.1 PerlinGenerator()

```
PerlinGenerator::PerlinGenerator () [inline]
```

Constructor for the PerlinGenerator. It initializes the permutation table.

6.6.2.2 ~PerlinGenerator()

```
PerlinGenerator::~PerlinGenerator () [inline]
```

Destructor for the PerlinGenerator. It deletes the permutation table.

6.6.3 Member Function Documentation

6.6.3.1 noise()

```
float PerlinGenerator::noise (
            float x,
            float y,
            ISmoothingFunction const & smoothing_function) [inline], [override], [virtual]
```

Generator for a given point a Perlin Noise.

Parameters

x	the X value of the point.
y	the Y value of the point.
smoothing_function	a smoothing function used to make the noise more regular.

Returns

The Perlin Noise value for the given point.

Implements NoiseGenerator.

6.6.4 Member Data Documentation

6.6.4.1 GRADIENT

```
const float PerlinGenerator::GRADIENT[8][2]
```

Initial value:

```
                    = {
    {TRANSLATION_UNIT,TRANSLATION_UNIT},
    {-TRANSLATION_UNIT,TRANSLATION_UNIT},
    {TRANSLATION_UNIT,-TRANSLATION_UNIT},
    {-TRANSLATION_UNIT,-TRANSLATION_UNIT},
    {1,0},
    {-1,0},
    {0,1},
    {0,-1}
}
```

6.6.4.2 permutation_table

```
int* PerlinGenerator::permutation_table
```

6.6.4.3 TRANSLATION_UNIT

```
const float PerlinGenerator::TRANSLATION_UNIT = 1.0f / std::sqrt(2.0f)
```

The documentation for this class was generated from the following file:

- src/noise/PerlinGenerator.h

6.7 PerlinMapGenerator Class Reference

```
#include <PerlinMapGenerator.h>
```

Public Member Functions

- PerlinMapGenerator (int octave=6, float persistence=0.5f, float lacunarity=2.0f, float scale=4.5f, float base_frequency=1.0f, float base_amplitude=1.0f, int width=1000, int height=1000)
- std::vector< std::vector< float > > generate (ISmoothingFunction const &smoothing)

Private Attributes

- int octave
- int width
- int height
- float persistence
- float lacunarity
- float scale
- float base_frequency
- float base_amplitude
- std::vector< std::vector< float > > perlin_values

6.7.1 Constructor & Destructor Documentation

6.7.1.1 PerlinMapGenerator()

```
PerlinMapGenerator::PerlinMapGenerator (
            int octave = 6,
            float persistence = 0.5f,
            float lacunarity = 2.0f,
            float scale = 4.5f,
            float base_frequency = 1.0f,
            float base_amplitude = 1.0f,
            int width = 1000,
            int height = 1000)  [inline]
```

6.7.2 Member Function Documentation

6.7.2.1 generate()

```
std::vector< std::vector< float > > PerlinMapGenerator::generate (
            ISmoothingFunction const & smoothing)  [inline]
```

6.7.3 Member Data Documentation

6.7.3.1 base_amplitude

```
float PerlinMapGenerator::base_amplitude  [private]
```

6.7.3.2 base_frequency

```
float PerlinMapGenerator::base_frequency  [private]
```

6.7.3.3 height

```
int PerlinMapGenerator::height  [private]
```

6.7.3.4 lacunarity

```
float PerlinMapGenerator::lacunarity  [private]
```

6.7.3.5 octave

```
int PerlinMapGenerator::octave  [private]
```

6.7.3.6 perlin_values

```
std::vector<std::vector<float> > PerlinMapGenerator::perlin_values  [private]
```

6.7.3.7 persistence

```
float PerlinMapGenerator::persistence  [private]
```

6.7.3.8 scale

```
float PerlinMapGenerator::scale  [private]
```

6.7.3.9 width

```
int PerlinMapGenerator::width [private]
```

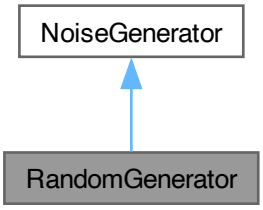
The documentation for this class was generated from the following file:

- src/map/PerlinMapGenerator.h

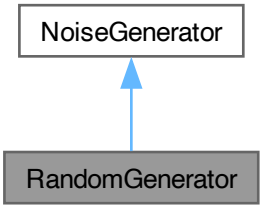
6.8 RandomGenerator Class Reference

```
#include <RandomGenerator.h>
```

Inheritance diagram for RandomGenerator:



Collaboration diagram for RandomGenerator:



Public Member Functions

- float noise (float x, float y, ISmoothingFunction const &smoothing_function) override

  A simple noise generator that returns a random value for each point.

Additional Inherited Members

Static Protected Member Functions inherited from NoiseGenerator

- static int * getPermutationTable ()

  Return a permutation table (lenght: 512).

6.8.1 Member Function Documentation

6.8.1.1 noise()

```
float RandomGenerator::noise (
            float x,
            float y,
            ISmoothingFunction const & smoothing_function)  [inline], [override], [virtual]
```

A simple noise generator that returns a random value for each point.

See also

NoiseGenerator

Author

Clément BARATIN

Generator for a given point a random value between 0 and 1.

Parameters

x	the X value of the point.
y	the Y value of the point.
smoothing_function	a smoothing function used to make the noise more regular.

Returns

the value of the point.

Implements NoiseGenerator.

The documentation for this class was generated from the following file:

- src/noise/RandomGenerator.h

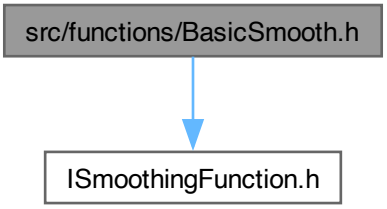
Chapter 7

File Documentation

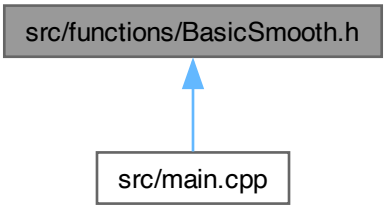
7.1 src/functions/BasicSmooth.h File Reference

`#include "ISmoothingFunction.h"`

Include dependency graph for BasicSmooth.h:



This graph shows which files directly or indirectly include this file:



Classes

- class BasicSmooth

  Implementation of smoothing function using polynomial values.

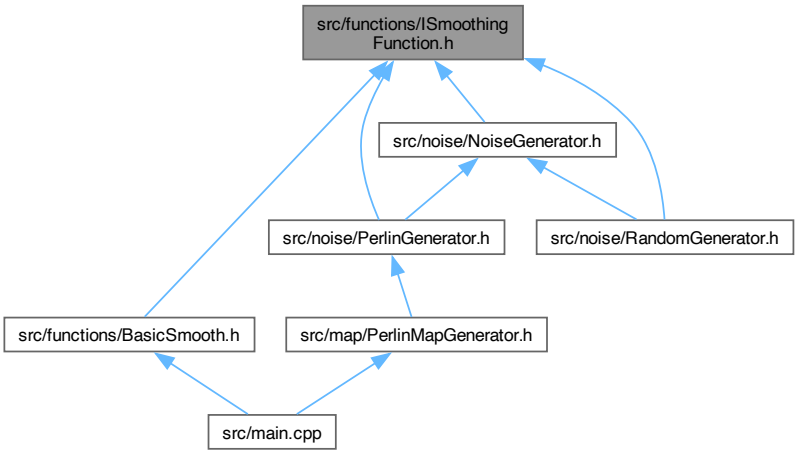
7.2 BasicSmooth.h

Go to the documentation of this file.

```
#pragma once

#include "ISmoothingFunction.h"

class BasicSmooth : public ISmoothingFunction {

public:

    float apply(float x) const override {
        return x * x * x * x * (x * (x * 6 - 15) + 10);
    }
};
```

7.3 src/functions/ISmoothingFunction.h File Reference

This graph shows which files directly or indirectly include this file:



Classes

- class ISmoothingFunction

  Interface to define a smoothing function.

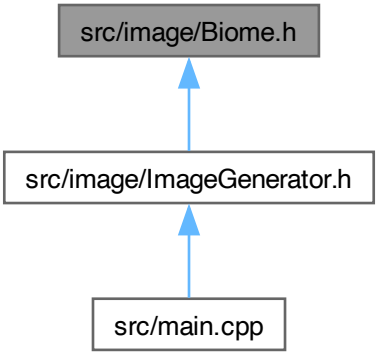
7.4 ISmoothingFunction.h

Go to the documentation of this file.

```
#pragma once

class ISmoothingFunction
{

public:
    virtual ~ISmoothingFunction() = default;
    virtual float apply(float x) const = 0;
};
```

7.5 src/image/Biome.h File Reference

This graph shows which files directly or indirectly include this file:



Classes

- struct BiomeColor

  Biome definitions and related functions for colorization.

Enumerations

- enum class Biome {
  DeepWater , ShallowWater , Reef , Shore ,
  Plains , Forest , Hills , Mountain ,
  Snow }

  Enumerate all possible biomes.

Functions

- Biome get_biome (float normalized)

  Get the biome object based on the normalized perlin value. Modifying the thresholds will change the distribution of the biomes on the generated image.
- BiomeColor get_biome_color (Biome biome, float normalized)

  Get the color associated with a biome. The color can be modified based on the normalized perlin value to add more variety to the image.

7.5.1 Enumeration Type Documentation

7.5.1.1 Biome

```
enum class Biome [strong]
```

Enumerate all possible biomes.

Author

Clément BARATIN

Enumerator

DeepWater	
ShallowWater	
Reef	
Shore	
Plains	
Forest	
Hills	
Mountain	
Snow	

7.5.2 Function Documentation

7.5.2.1 get_biome()

```
Biome get_biome (
            float normalized) [inline]
```

Get the biome object based on the normalized perlin value. Modifying the thresholds will change the distribution of the biomes on the generated image.

Parameters

normalized	

Returns

Biome

7.5.2.2 get_biome_color()

```
BiomeColor get_biome_color (
            Biome biome,
            float normalized)  [inline]
```

Get the color associated with a biome. The color can be modified based on the normalized perlin value to add more variety to the image.

Parameters

biome	the biome for which we want the color.
normalized	the normalized perlin value, used to add variety to the colorization.

Returns

BiomeColor

7.6 Biome.h

Go to the documentation of this file.

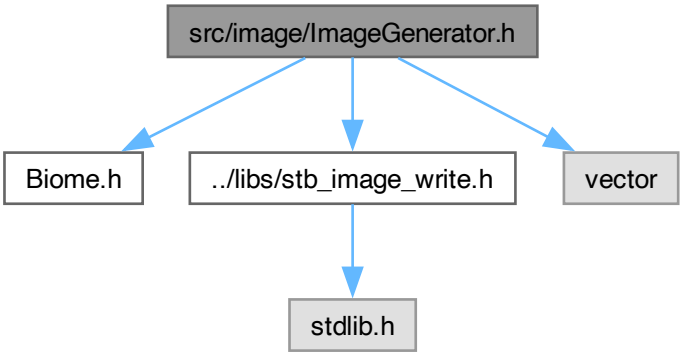
```
#pragma once

struct BiomeColor {
    unsigned char r;
    unsigned char g;
    unsigned char b;
};

enum class Biome {
    DeepWater,
    ShallowWater,
    Reef,
    Shore,
    Plains,
    Forest,
    Hills,
    Mountain,
    Snow,
};

inline Biome get_biome(float normalized) {
    if (normalized < 0.32f) {
        return Biome::DeepWater;
    }
    if (normalized < 0.42f) {
        return Biome::ShallowWater;
    }
    if (normalized < 0.45f) {
        return Biome::Reef;
    }
    if (normalized < 0.48f) {
        return Biome::Shore;
    }
    if (normalized < 0.62f) {
        return Biome::Plains;
    }
    if (normalized < 0.78f) {
        return Biome::Forest;
    }
    if (normalized < 0.84f) {
        return Biome::Hills;
    }
    if (normalized < 0.94f) {
        return Biome::Mountain;
    }
    return Biome::Snow;
}

inline BiomeColor get_biome_color(Biome biome, float normalized) {
```

```
    switch (biome) {
        case Biome::DeepWater:
            return {8, 36, 108};
        case Biome::ShallowWater:
            return {28, 96, 168};
        case Biome::Reef:
            return {72, 170, 200};
        case Biome::Shore:
            return {214, 201, 155};
        case Biome::Plains:
            return {146, 168, 92};
        case Biome::Forest:
            return {52, 110, 56};
        case Biome::Hills:
            return {122, 124, 96};
        case Biome::Mountain:
            return {108, 108, 110};
        case Biome::Snow:
        default:
            if (normalized < 0.93f) {
                return {212, 212, 212};
            }
            return {246, 246, 246};
    }
}
```

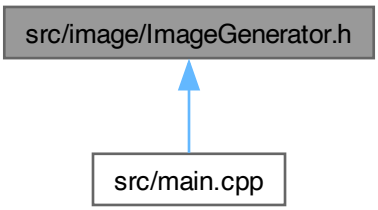
7.7 src/image/ImageGenerator.h File Reference

```
#include "Biome.h"
#include "../libs/stb_image_write.h"
#include <vector>
```

Include dependency graph for ImageGenerator.h:



This graph shows which files directly or indirectly include this file:



Classes

- class ImageGenerator

  Generate an image based on a Perlin Map.

7.8 ImageGenerator.h

Go to the documentation of this file.

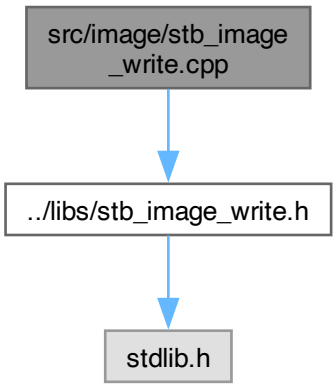
```
#pragma once

#include "Biome.h"
#include "../libs/stb_image_write.h"
#include <vector>

class ImageGenerator {

public:

    static void generate_png_image(
        std::vector<std::vector<float» perlin_value,
        int width,
        int height,
        const char* filename,
        bool colorize = false
    ) {

        const int channels = 3;
        std::vector<unsigned char> pixels(width * height * channels);

        for (int i = 0; i < width; i++)
        {

            for (int j = 0; j < height; j++)
            {
                float value = perlin_value[i][j];
                float normalized = (value + 1.0f) * 0.5f;

                if (normalized < 0.0f) normalized = 0.0f;
                if (normalized > 1.0f) normalized = 1.0f;

                if (colorize)
                {
                    const Biome biome = get_biome(normalized);
                    const BiomeColor color = get_biome_color(biome, normalized);
                    ImageGenerator::set_pixel(pixels, width, i, j, color.r, color.g, color.b);
                }
                else
                {
                    unsigned char v = static_cast<unsigned char>(normalized * 255.0f);
                    ImageGenerator::set_pixel(pixels, width, i, j, v, v, v);
                }
            }
```

```
        }

        stbi_write_png(filename, width, height, channels, pixels.data(), width * channels);
    }

    static void set_pixel(std::vector<unsigned char>& pixels, int width, int x, int y, unsigned char
    r, unsigned char g, unsigned char b) {
        const int channels = 3;
        const int idx = (y * width + x) * channels;
        pixels[idx + 0] = r;
        pixels[idx + 1] = g;
        pixels[idx + 2] = b;
    }
};
```

7.9 src/image/stb_image_write.cpp File Reference

`#include "../libs/stb_image_write.h"`

Include dependency graph for stb_image_write.cpp:



Macros

- #define STB_IMAGE_WRITE_IMPLEMENTATION

7.9.1 Macro Definition Documentation

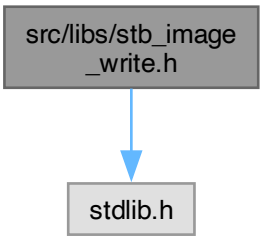
7.9.1.1 STB_IMAGE_WRITE_IMPLEMENTATION

`#define STB_IMAGE_WRITE_IMPLEMENTATION`

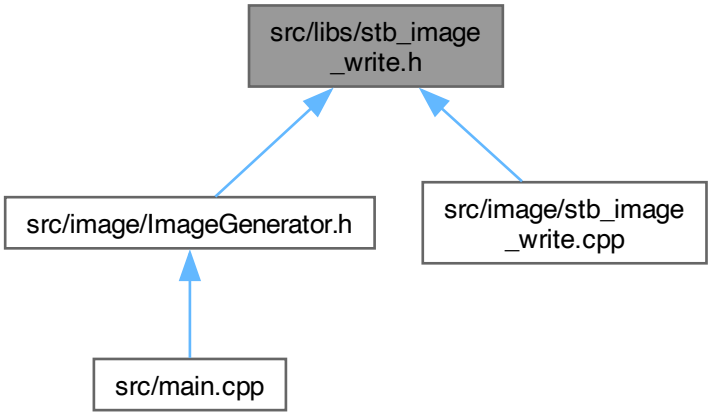
7.10 src/libs/stb_image_write.h File Reference

```
#include <stdlib.h>
```

Include dependency graph for stb_image_write.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define STBIWDEF extern

Typedefs

- typedef void stbi_write_func(void ∗context, void ∗data, int size)

Functions

- STBIWDEF int stbi_write_png (char const ∗filename, int w, int h, int comp, const void ∗data, int stride_in_↩ bytes)
- STBIWDEF int stbi_write_bmp (char const ∗filename, int w, int h, int comp, const void ∗data)
- STBIWDEF int stbi_write_tga (char const ∗filename, int w, int h, int comp, const void ∗data)
- STBIWDEF int stbi_write_hdr (char const ∗filename, int w, int h, int comp, const float ∗data)
- STBIWDEF int stbi_write_jpg (char const ∗filename, int x, int y, int comp, const void ∗data, int quality)
- STBIWDEF int stbi_write_png_to_func (stbi_write_func ∗func, void ∗context, int w, int h, int comp, const void ∗data, int stride_in_bytes)
- STBIWDEF int stbi_write_bmp_to_func (stbi_write_func ∗func, void ∗context, int w, int h, int comp, const void ∗data)
- STBIWDEF int stbi_write_tga_to_func (stbi_write_func ∗func, void ∗context, int w, int h, int comp, const void ∗data)
- STBIWDEF int stbi_write_hdr_to_func (stbi_write_func ∗func, void ∗context, int w, int h, int comp, const float ∗data)
- STBIWDEF int stbi_write_jpg_to_func (stbi_write_func ∗func, void ∗context, int x, int y, int comp, const void ∗data, int quality)
- STBIWDEF void stbi_flip_vertically_on_write (int flip_boolean)

Variables

- STBIWDEF int stbi_write_tga_with_rle
- STBIWDEF int stbi_write_png_compression_level
- STBIWDEF int stbi_write_force_png_filter

7.10.1 Macro Definition Documentation

7.10.1.1 STBIWDEF

```
#define STBIWDEF extern
```

7.10.2 Typedef Documentation

7.10.2.1 stbi_write_func

```
typedef void stbi_write_func(void *context, void *data, int size)
```

7.10.3 Function Documentation

7.10.3.1 stbi_flip_vertically_on_write()

```
STBIWDEF void stbi_flip_vertically_on_write (
            int flip_boolean)
```

7.10.3.2 stbi_write_bmp()

```
STBIWDEF int stbi_write_bmp (
            char const * filename,
            int w,
            int h,
            int comp,
            const void * data)
```

7.10.3.3 stbi_write_bmp_to_func()

```
STBIWDEF int stbi_write_bmp_to_func (
            stbi_write_func * func,
            void * context,
            int w,
            int h,
            int comp,
            const void * data)
```

7.10.3.4 stbi_write_hdr()

```
STBIWDEF int stbi_write_hdr (
            char const * filename,
            int w,
            int h,
            int comp,
            const float * data)
```

7.10.3.5 stbi_write_hdr_to_func()

```
STBIWDEF int stbi_write_hdr_to_func (
            stbi_write_func * func,
            void * context,
            int w,
            int h,
            int comp,
            const float * data)
```

7.10.3.6 stbi_write_jpg()

```
STBIWDEF int stbi_write_jpg (
            char const * filename,
            int x,
            int y,
            int comp,
            const void * data,
            int quality)
```

7.10.3.7 stbi_write_jpg_to_func()

```
STBIWDEF int stbi_write_jpg_to_func (
            stbi_write_func * func,
            void * context,
            int x,
            int y,
            int comp,
            const void * data,
            int quality)
```

7.10.3.8 stbi_write_png()

```
STBIWDEF int stbi_write_png (
            char const * filename,
            int w,
            int h,
            int comp,
            const void * data,
            int stride_in_bytes)
```

7.10.3.9 stbi_write_png_to_func()

```
STBIWDEF int stbi_write_png_to_func (
            stbi_write_func * func,
            void * context,
            int w,
            int h,
            int comp,
            const void * data,
            int stride_in_bytes)
```

7.10.3.10 stbi_write_tga()

```
STBIWDEF int stbi_write_tga (
            char const * filename,
            int w,
            int h,
            int comp,
            const void * data)
```

7.10.3.11 stbi_write_tga_to_func()

```
STBIWDEF int stbi_write_tga_to_func (
            stbi_write_func * func,
            void * context,
            int w,
            int h,
            int comp,
            const void * data)
```

7.10.4 Variable Documentation

7.10.4.1 stbi_write_force_png_filter

```
STBIWDEF int stbi_write_force_png_filter
```

7.10.4.2 stbi_write_png_compression_level

```
STBIWDEF int stbi_write_png_compression_level
```

7.10.4.3 stbi_write_tga_with_rle

```
STBIWDEF int stbi_write_tga_with_rle
```

7.11 stb_image_write.h

Go to the documentation of this file.

```
/* stb_image_write - v1.16 - public domain - http://nothings.org/stb
   writes out PNG/BMP/TGA/JPEG/HDR images to C stdio - Sean Barrett 2010-2015
                                     no warranty implied; use at your own risk

   Before #including,

       #define STB_IMAGE_WRITE_IMPLEMENTATION

   in the file that you want to have the implementation.

   Will probably not work correctly with strict-aliasing optimizations.

ABOUT:

   This header file is a library for writing images to C stdio or a callback.

   The PNG output is not optimal; it is 20-50% larger than the file
   written by a decent optimizing implementation; though providing a custom
   zlib compress function (see STBIW_ZLIB_COMPRESS) can mitigate that.
   This library is designed for source code compactness and simplicity,
   not optimal image file size or run-time performance.

BUILDING:

   You can #define STBIW_ASSERT(x) before the #include to avoid using assert.h.
   You can #define STBIW_MALLOC(), STBIW_REALLOC(), and STBIW_FREE() to replace
   malloc,realloc,free.
   You can #define STBIW_MEMMOVE() to replace memmove()
   You can #define STBIW_ZLIB_COMPRESS to use a custom zlib-style compress function
   for PNG compression (instead of the builtin one), it must have the following signature:
   unsigned char * my_compress(unsigned char *data, int data_len, int *out_len, int quality);
   The returned data will be freed with STBIW_FREE() (free() by default),
   so it must be heap allocated with STBIW_MALLOC() (malloc() by default),

UNICODE:

   If compiling for Windows and you wish to use Unicode filenames, compile
   with
       #define STBIW_WINDOWS_UTF8
   and pass utf8-encoded filenames. Call stbiw_convert_wchar_to_utf8 to convert
   Windows wchar_t filenames to utf8.

USAGE:

   There are five functions, one for each image file format:

     int stbi_write_png(char const *filename, int w, int h, int comp, const void *data, int stride_in_bytes);
     int stbi_write_bmp(char const *filename, int w, int h, int comp, const void *data);
     int stbi_write_tga(char const *filename, int w, int h, int comp, const void *data);
     int stbi_write_jpg(char const *filename, int w, int h, int comp, const void *data, int quality);
```

```
     int stbi_write_hdr(char const *filename, int w, int h, int comp, const float *data);

     void stbi_flip_vertically_on_write(int flag); // flag is non-zero to flip data vertically

   There are also five equivalent functions that use an arbitrary write function. You are
   expected to open/close your file-equivalent before and after calling these:

     int stbi_write_png_to_func(stbi_write_func *func, void *context, int w, int h, int comp, const void  *data, int stride_in_bytes);
     int stbi_write_bmp_to_func(stbi_write_func *func, void *context, int w, int h, int comp, const void  *data);
     int stbi_write_tga_to_func(stbi_write_func *func, void *context, int w, int h, int comp, const void  *data);
     int stbi_write_hdr_to_func(stbi_write_func *func, void *context, int w, int h, int comp, const float *data);
     int stbi_write_jpg_to_func(stbi_write_func *func, void *context, int x, int y, int comp, const void *data, int quality);

   where the callback is:
      void stbi_write_func(void *context, void *data, int size);

   You can configure it with these global variables:
      int stbi_write_tga_with_rle;             // defaults to true; set to 0 to disable RLE
      int stbi_write_png_compression_level;    // defaults to 8; set to higher for more compression
      int stbi_write_force_png_filter;         // defaults to -1; set to 0..5 to force a filter mode


   You can define STBI_WRITE_NO_STDIO to disable the file variant of these
   functions, so the library will not use stdio.h at all. However, this will
   also disable HDR writing, because it requires stdio for formatted output.

   Each function returns 0 on failure and non-0 on success.

   The functions create an image file defined by the parameters. The image
   is a rectangle of pixels stored from left-to-right, top-to-bottom.
   Each pixel contains 'comp' channels of data stored interleaved with 8-bits
   per channel, in the following order: 1=Y, 2=YA, 3=RGB, 4=RGBA. (Y is
   monochrome color.) The rectangle is 'w' pixels wide and 'h' pixels tall.
   The *data pointer points to the first byte of the top-left-most pixel.
   For PNG, "stride_in_bytes" is the distance in bytes from the first byte of
   a row of pixels to the first byte of the next row of pixels.

   PNG creates output files with the same number of components as the input.
   The BMP format expands Y to RGB in the file format and does not
   output alpha.

   PNG supports writing rectangles of data even when the bytes storing rows of
   data are not consecutive in memory (e.g. sub-rectangles of a larger image),
   by supplying the stride between the beginning of adjacent rows. The other
   formats do not. (Thus you cannot write a native-format BMP through the BMP
   writer, both because it is in BGR order and because it may have padding
   at the end of the line.)

   PNG allows you to set the deflate compression level by setting the global
   variable 'stbi_write_png_compression_level' (it defaults to 8).

   HDR expects linear float data. Since the format is always 32-bit rgb(e)
   data, alpha (if provided) is discarded, and for monochrome data it is
   replicated across all three channels.

   TGA supports RLE or non-RLE compressed data. To use non-RLE-compressed
   data, set the global variable 'stbi_write_tga_with_rle' to 0.

   JPEG does ignore alpha channels in input data; quality is between 1 and 100.
   Higher quality looks better but results in a bigger image.
   JPEG baseline (no JPEG progressive).

CREDITS:


   Sean Barrett           -    PNG/BMP/TGA
   Baldur Karlsson        -    HDR
   Jean-Sebastien Guay    -    TGA monochrome
   Tim Kelsey             -    misc enhancements
   Alan Hickman           -    TGA RLE
   Emmanuel Julien        -    initial file IO callback implementation
   Jon Olick              -    original jo_jpeg.cpp code
   Daniel Gibson          -    integrate JPEG, allow external zlib
   Aarni Koskela          -    allow choosing PNG filter

   bugfixes:
      github:Chribba
      Guillaume Chereau
      github:jry2
      github:romigrou
      Sergio Gonzalez
      Jonas Karlsson
```

```
      Filip Wasil
      Thatcher Ulrich
      github:poppolopoppo
      Patrick Boettcher
      github:xeekworx
      Cap Petschulat
      Simon Rodriguez
      Ivan Tikhonov
      github:ignotion
      Adam Schackart
      Andrew Kensler

LICENSE

  See end of file for license information.

*/

#ifndef INCLUDE_STB_IMAGE_WRITE_H
#define INCLUDE_STB_IMAGE_WRITE_H

#include <stdlib.h>

// if STB_IMAGE_WRITE_STATIC causes problems, try defining STBIWDEF to 'inline' or 'static inline'
#ifndef STBIWDEF
#ifdef STB_IMAGE_WRITE_STATIC
#define STBIWDEF  static
#else
#ifdef __cplusplus
#define STBIWDEF  extern "C"
#else
#define STBIWDEF  extern
#endif
#endif
#endif

#ifndef STB_IMAGE_WRITE_STATIC  // C++ forbids static forward declarations
STBIWDEF int stbi_write_tga_with_rle;
STBIWDEF int stbi_write_png_compression_level;
STBIWDEF int stbi_write_force_png_filter;
#endif

#ifndef STBI_WRITE_NO_STDIO
STBIWDEF int stbi_write_png(char const *filename, int w, int h, int comp, const void  *data, int stride_in_bytes);
STBIWDEF int stbi_write_bmp(char const *filename, int w, int h, int comp, const void  *data);
STBIWDEF int stbi_write_tga(char const *filename, int w, int h, int comp, const void  *data);
STBIWDEF int stbi_write_hdr(char const *filename, int w, int h, int comp, const float *data);
STBIWDEF int stbi_write_jpg(char const *filename, int x, int y, int comp, const void  *data, int quality);

#ifdef STBIW_WINDOWS_UTF8
STBIWDEF int stbiw_convert_wchar_to_utf8(char *buffer, size_t bufferlen, const wchar_t* input);
#endif
#endif

typedef void stbi_write_func(void *context, void *data, int size);

STBIWDEF int stbi_write_png_to_func(stbi_write_func *func, void *context, int w, int h, int comp, const void  *data, int stride_in_bytes);
STBIWDEF int stbi_write_bmp_to_func(stbi_write_func *func, void *context, int w, int h, int comp, const void  *data);
STBIWDEF int stbi_write_tga_to_func(stbi_write_func *func, void *context, int w, int h, int comp, const void  *data);
STBIWDEF int stbi_write_hdr_to_func(stbi_write_func *func, void *context, int w, int h, int comp, const float *data);
STBIWDEF int stbi_write_jpg_to_func(stbi_write_func *func, void *context, int x, int y, int comp, const void  *data, int quality);

STBIWDEF void stbi_flip_vertically_on_write(int flip_boolean);

#endif//INCLUDE_STB_IMAGE_WRITE_H

#ifdef STB_IMAGE_WRITE_IMPLEMENTATION

#ifdef _WIN32
   #ifndef _CRT_SECURE_NO_WARNINGS
   #define _CRT_SECURE_NO_WARNINGS
   #endif
   #ifndef _CRT_NONSTDC_NO_DEPRECATE
   #define _CRT_NONSTDC_NO_DEPRECATE
   #endif
#endif

#ifndef STBI_WRITE_NO_STDIO
#include <stdio.h>
#endif // STBI_WRITE_NO_STDIO
```

```

#include <stdarg.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#if defined(STBIW_MALLOC) && defined(STBIW_FREE) && (defined(STBIW_REALLOC) ||
defined(STBIW_REALLOC_SIZED))
// ok
#elif !defined(STBIW_MALLOC) && !defined(STBIW_FREE) && !defined(STBIW_REALLOC) &&
!defined(STBIW_REALLOC_SIZED)
// ok
#else
#error "Must define all or none of STBIW_MALLOC, STBIW_FREE, and STBIW_REALLOC (or
STBIW_REALLOC_SIZED)."
#endif

#ifndef STBIW_MALLOC
#define STBIW_MALLOC(sz)        malloc(sz)
#define STBIW_REALLOC(p,newsz)  realloc(p,newsz)
#define STBIW_FREE(p)           free(p)
#endif

#ifndef STBIW_REALLOC_SIZED
#define STBIW_REALLOC_SIZED(p,oldsz,newsz) STBIW_REALLOC(p,newsz)
#endif


#ifndef STBIW_MEMMOVE
#define STBIW_MEMMOVE(a,b,sz) memmove(a,b,sz)
#endif


#ifndef STBIW_ASSERT
#include <assert.h>
#define STBIW_ASSERT(x) assert(x)
#endif

#define STBIW_UCHAR(x) (unsigned char) ((x) & 0xff)

#ifdef STB_IMAGE_WRITE_STATIC
static int stbi_write_png_compression_level = 8;
static int stbi_write_tga_with_rle = 1;
static int stbi_write_force_png_filter = -1;
#else
int stbi_write_png_compression_level = 8;
int stbi_write_tga_with_rle = 1;
int stbi_write_force_png_filter = -1;
#endif

static int stbi__flip_vertically_on_write = 0;

STBIWDEF void stbi_flip_vertically_on_write(int flag)
{
   stbi__flip_vertically_on_write = flag;
}

typedef struct
{
   stbi_write_func *func;
   void *context;
   unsigned char buffer[64];
   int buf_used;
} stbi__write_context;

// initialize a callback-based context
static void stbi__start_write_callbacks(stbi__write_context *s, stbi_write_func *c, void *context)
{
   s->func    = c;
   s->context = context;
}

#ifndef STBI_WRITE_NO_STDIO

static void stbi__stdio_write(void *context, void *data, int size)
{
   fwrite(data,1,size,(FILE*) context);
}

#if defined(_WIN32) && defined(STBIW_WINDOWS_UTF8)
#ifdef __cplusplus
#define STBIW_EXTERN extern "C"
#else
#define STBIW_EXTERN extern
#endif
STBIW_EXTERN __declspec(dllimport) int __stdcall MultiByteToWideChar(unsigned int cp, unsigned long
flags, const char *str, int cbmb, wchar_t *widestr, int cchwide);
```

```
STBIW_EXTERN __declspec(dllimport) int __stdcall WideCharToMultiByte(unsigned int cp, unsigned long flags, const wchar_t *widestr, int cchwide, char *str, int cbmb, const char *defchar, int *used_default);

STBIWDEF int stbiw_convert_wchar_to_utf8(char *buffer, size_t bufferlen, const wchar_t* input)
{
   return WideCharToMultiByte(65001 /* UTF8 */, 0, input, -1, buffer, (int) bufferlen, NULL, NULL);
}
#endif

static FILE *stbiw__fopen(char const *filename, char const *mode)
{
   FILE *f;
#if defined(_WIN32) && defined(STBIW_WINDOWS_UTF8)
   wchar_t wMode[64];
   wchar_t wFilename[1024];
   if (0 == MultiByteToWideChar(65001 /* UTF8 */, 0, filename, -1, wFilename, sizeof(wFilename)/sizeof(*wFilename)))
      return 0;

   if (0 == MultiByteToWideChar(65001 /* UTF8 */, 0, mode, -1, wMode, sizeof(wMode)/sizeof(*wMode)))
      return 0;

#if defined(_MSC_VER) && _MSC_VER >= 1400
   if (0 != _wfopen_s(&f, wFilename, wMode))
      f = 0;
#else
   f = _wfopen(wFilename, wMode);
#endif

#elif defined(_MSC_VER) && _MSC_VER >= 1400
   if (0 != fopen_s(&f, filename, mode))
      f=0;
#else
   f = fopen(filename, mode);
#endif
   return f;
}

static int stbi__start_write_file(stbi__write_context *s, const char *filename)
{
   FILE *f = stbiw__fopen(filename, "wb");
   stbi__start_write_callbacks(s, stbi__stdio_write, (void *) f);
   return f != NULL;
}

static void stbi__end_write_file(stbi__write_context *s)
{
   fclose((FILE *)s->context);
}

#endif // !STBI_WRITE_NO_STDIO

typedef unsigned int stbiw_uint32;
typedef int stb_image_write_test[sizeof(stbiw_uint32)==4 ? 1 : -1];

static void stbiw__writefv(stbi__write_context *s, const char *fmt, va_list v)
{
   while (*fmt) {
      switch (*fmt++) {
         case ' ': break;
         case '1': { unsigned char x = STBIW_UCHAR(va_arg(v, int));
                     s->func(s->context,&x,1);
                     break; }
         case '2': { int x = va_arg(v,int);
                     unsigned char b[2];
                     b[0] = STBIW_UCHAR(x);
                     b[1] = STBIW_UCHAR(x>>8);
                     s->func(s->context,b,2);
                     break; }
         case '4': { stbiw_uint32 x = va_arg(v,int);
                     unsigned char b[4];
                     b[0]=STBIW_UCHAR(x);
                     b[1]=STBIW_UCHAR(x>>8);
                     b[2]=STBIW_UCHAR(x>>16);
                     b[3]=STBIW_UCHAR(x>>24);
                     s->func(s->context,b,4);
                     break; }
         default:
            STBIW_ASSERT(0);
            return;
      }
   }
}

static void stbiw__writef(stbi__write_context *s, const char *fmt, ...)
{
```

```
   va_list v;
   va_start(v, fmt);
   stbiw__writefv(s, fmt, v);
   va_end(v);
}

static void stbiw__write_flush(stbi__write_context *s)
{
   if (s->buf_used) {
      s->func(s->context, &s->buffer, s->buf_used);
      s->buf_used = 0;
   }
}

static void stbiw__putc(stbi__write_context *s, unsigned char c)
{
   s->func(s->context, &c, 1);
}

static void stbiw__write1(stbi__write_context *s, unsigned char a)
{
   if ((size_t)s->buf_used + 1 > sizeof(s->buffer))
      stbiw__write_flush(s);
   s->buffer[s->buf_used++] = a;
}

static void stbiw__write3(stbi__write_context *s, unsigned char a, unsigned char b, unsigned char c)
{
   int n;
   if ((size_t)s->buf_used + 3 > sizeof(s->buffer))
      stbiw__write_flush(s);
   n = s->buf_used;
   s->buf_used = n+3;
   s->buffer[n+0] = a;
   s->buffer[n+1] = b;
   s->buffer[n+2] = c;
}

static void stbiw__write_pixel(stbi__write_context *s, int rgb_dir, int comp, int write_alpha, int
expand_mono, unsigned char *d)
{
   unsigned char bg[3] = { 255, 0, 255}, px[3];
   int k;

   if (write_alpha < 0)
      stbiw__write1(s, d[comp - 1]);

   switch (comp) {
      case 2: // 2 pixels = mono + alpha, alpha is written separately, so same as 1-channel case
      case 1:
         if (expand_mono)
            stbiw__write3(s, d[0], d[0], d[0]); // monochrome bmp
         else
            stbiw__write1(s, d[0]);  // monochrome TGA
         break;
      case 4:
         if (!write_alpha) {
            // composite against pink background
            for (k = 0; k < 3; ++k)
               px[k] = bg[k] + ((d[k] - bg[k]) * d[3]) / 255;
            stbiw__write3(s, px[1 - rgb_dir], px[1], px[1 + rgb_dir]);
            break;
         }
         /* FALLTHROUGH */
      case 3:
         stbiw__write3(s, d[1 - rgb_dir], d[1], d[1 + rgb_dir]);
         break;
   }
   if (write_alpha > 0)
      stbiw__write1(s, d[comp - 1]);
}

static void stbiw__write_pixels(stbi__write_context *s, int rgb_dir, int vdir, int x, int y, int comp,
void *data, int write_alpha, int scanline_pad, int expand_mono)
{
   stbiw_uint32 zero = 0;
   int i,j, j_end;

   if (y <= 0)
      return;

   if (stbi__flip_vertically_on_write)
      vdir *= -1;

   if (vdir < 0) {
      j_end = -1; j = y-1;
   } else {
```

```
      j_end =  y; j = 0;
   }

   for (; j != j_end; j += vdir) {
      for (i=0; i < x; ++i) {
         unsigned char *d = (unsigned char *) data + (j*x+i)*comp;
         stbiw__write_pixel(s, rgb_dir, comp, write_alpha, expand_mono, d);
      }
      stbiw__write_flush(s);
      s->func(s->context, &zero, scanline_pad);
   }
}

static int stbiw__outfile(stbi__write_context *s, int rgb_dir, int vdir, int x, int y, int comp, int expand_mono, void *data, int alpha, int pad, const char *fmt, ...)
{
   if (y < 0 || x < 0) {
      return 0;
   } else {
      va_list v;
      va_start(v, fmt);
      stbiw__writefv(s, fmt, v);
      va_end(v);
      stbiw__write_pixels(s,rgb_dir,vdir,x,y,comp,data,alpha,pad, expand_mono);
      return 1;
   }
}

static int stbi_write_bmp_core(stbi__write_context *s, int x, int y, int comp, const void *data)
{
   if (comp != 4) {
      // write RGB bitmap
      int pad = (-x*3) & 3;
      return stbiw__outfile(s,-1,-1,x,y,comp,1,(void *) data,0,pad,
              "11 4 22 4" "4 44 22 444444",
              'B', 'M', 14+40+(x*3+pad)*y, 0,0, 14+40,  // file header
               40, x,y, 1,24, 0,0,0,0,0,0);             // bitmap header
   } else {
      // RGBA bitmaps need a v4 header
      // use BI_BITFIELDS mode with 32bpp and alpha mask
      // (straight BI_RGB with alpha mask doesn't work in most readers)
      return stbiw__outfile(s,-1,-1,x,y,comp,1,(void *)data,1,0,
         "11 4 22 4" "4 44 22 444444 4444 4 444 444 444 444",
         'B', 'M', 14+108+x*y*4, 0, 0, 14+108, // file header
         108, x,y, 1,32, 3,0,0,0,0,0, 0xff0000,0xff00,0xff,0xff000000u, 0, 0,0,0, 0,0,0, 0,0,0, 0,0,0); // bitmap V4 header
   }
}

STBIWDEF int stbi_write_bmp_to_func(stbi_write_func *func, void *context, int x, int y, int comp, const void *data)
{
   stbi__write_context s = { 0 };
   stbi__start_write_callbacks(&s, func, context);
   return stbi_write_bmp_core(&s, x, y, comp, data);
}

#ifndef STBI_WRITE_NO_STDIO
STBIWDEF int stbi_write_bmp(char const *filename, int x, int y, int comp, const void *data)
{
   stbi__write_context s = { 0 };
   if (stbi__start_write_file(&s,filename)) {
      int r = stbi_write_bmp_core(&s, x, y, comp, data);
      stbi__end_write_file(&s);
      return r;
   } else
      return 0;
}
#endif

static int stbi_write_tga_core(stbi__write_context *s, int x, int y, int comp, void *data)
{
   int has_alpha = (comp == 2 || comp == 4);
   int colorbytes = has_alpha ? comp-1 : comp;
   int format = colorbytes < 2 ? 3 : 2; // 3 color channels (RGB/RGBA) = 2, 1 color channel (Y/YA) = 3

   if (y < 0 || x < 0)
      return 0;

   if (!stbi_write_tga_with_rle) {
      return stbiw__outfile(s, -1, -1, x, y, comp, 0, (void *) data, has_alpha, 0,
         "111 221 2222 11", 0, 0, format, 0, 0, 0, 0, 0, x, y, (colorbytes + has_alpha) * 8, has_alpha * 8);
   } else {
      int i,j,k;
      int jend, jdir;

```

```
      stbiw__writef(s, "111 221 2222 11", 0,0,format+8, 0,0,0, 0,0,x,y, (colorbytes + has_alpha) * 8, has_alpha * 8);

      if (stbi__flip_vertically_on_write) {
         j = 0;
         jend = y;
         jdir = 1;
      } else {
         j = y-1;
         jend = -1;
         jdir = -1;
      }
      for (; j != jend; j += jdir) {
         unsigned char *row = (unsigned char *) data + j * x * comp;
         int len;

         for (i = 0; i < x; i += len) {
            unsigned char *begin = row + i * comp;
            int diff = 1;
            len = 1;

            if (i < x - 1) {
               ++len;
               diff = memcmp(begin, row + (i + 1) * comp, comp);
               if (diff) {
                  const unsigned char *prev = begin;
                  for (k = i + 2; k < x && len < 128; ++k) {
                     if (memcmp(prev, row + k * comp, comp)) {
                        prev += comp;
                        ++len;
                     } else {
                        --len;
                        break;
                     }
                  }
               } else {
                  for (k = i + 2; k < x && len < 128; ++k) {
                     if (!memcmp(begin, row + k * comp, comp)) {
                        ++len;
                     } else {
                        break;
                     }
                  }
               }
            }

            if (diff) {
               unsigned char header = STBIW_UCHAR(len - 1);
               stbiw__write1(s, header);
               for (k = 0; k < len; ++k) {
                  stbiw__write_pixel(s, -1, comp, has_alpha, 0, begin + k * comp);
               }
            } else {
               unsigned char header = STBIW_UCHAR(len - 129);
               stbiw__write1(s, header);
               stbiw__write_pixel(s, -1, comp, has_alpha, 0, begin);
            }
         }
      }
      stbiw__write_flush(s);
   }
   return 1;
}

STBIWDEF int stbi_write_tga_to_func(stbi_write_func *func, void *context, int x, int y, int comp, const void *data)
{
   stbi__write_context s = { 0 };
   stbi__start_write_callbacks(&s, func, context);
   return stbi_write_tga_core(&s, x, y, comp, (void *) data);
}

#ifndef STBI_WRITE_NO_STDIO
STBIWDEF int stbi_write_tga(char const *filename, int x, int y, int comp, const void *data)
{
   stbi__write_context s = { 0 };
   if (stbi__start_write_file(&s,filename)) {
      int r = stbi_write_tga_core(&s, x, y, comp, (void *) data);
      stbi__end_write_file(&s);
      return r;
   } else
      return 0;
}
#endif

// ***********************************************************************************************
// Radiance RGBE HDR writer
```

```
// by Baldur Karlsson

#define stbiw__max(a, b)  ((a) > (b) ? (a) : (b))

#ifndef STBI_WRITE_NO_STDIO

static void stbiw__linear_to_rgbe(unsigned char *rgbe, float *linear)
{
   int exponent;
   float maxcomp = stbiw__max(linear[0], stbiw__max(linear[1], linear[2]));

   if (maxcomp < 1e-32f) {
      rgbe[0] = rgbe[1] = rgbe[2] = rgbe[3] = 0;
   } else {
      float normalize = (float) frexp(maxcomp, &exponent) * 256.0f/maxcomp;

      rgbe[0] = (unsigned char)(linear[0] * normalize);
      rgbe[1] = (unsigned char)(linear[1] * normalize);
      rgbe[2] = (unsigned char)(linear[2] * normalize);
      rgbe[3] = (unsigned char)(exponent + 128);
   }
}

static void stbiw__write_run_data(stbi__write_context *s, int length, unsigned char databyte)
{
   unsigned char lengthbyte = STBIW_UCHAR(length+128);
   STBIW_ASSERT(length+128 <= 255);
   s->func(s->context, &lengthbyte, 1);
   s->func(s->context, &databyte, 1);
}

static void stbiw__write_dump_data(stbi__write_context *s, int length, unsigned char *data)
{
   unsigned char lengthbyte = STBIW_UCHAR(length);
   STBIW_ASSERT(length <= 128); // inconsistent with spec but consistent with official code
   s->func(s->context, &lengthbyte, 1);
   s->func(s->context, data, length);
}

static void stbiw__write_hdr_scanline(stbi__write_context *s, int width, int ncomp, unsigned char
*scratch, float *scanline)
{
   unsigned char scanlineheader[4] = { 2, 2, 0, 0 };
   unsigned char rgbe[4];
   float linear[3];
   int x;

   scanlineheader[2] = (width&0xff00)>>8;
   scanlineheader[3] = (width&0x00ff);

   /* skip RLE for images too small or large */
   if (width < 8 || width >= 32768) {
      for (x=0; x < width; x++) {
         switch (ncomp) {
            case 4: /* fallthrough */
            case 3: linear[2] = scanline[x*ncomp + 2];
                    linear[1] = scanline[x*ncomp + 1];
                    linear[0] = scanline[x*ncomp + 0];
                    break;
            default:
                    linear[0] = linear[1] = linear[2] = scanline[x*ncomp + 0];
                    break;
         }
         stbiw__linear_to_rgbe(rgbe, linear);
         s->func(s->context, rgbe, 4);
      }
   } else {
      int c,r;
      /* encode into scratch buffer */
      for (x=0; x < width; x++) {
         switch(ncomp) {
            case 4: /* fallthrough */
            case 3: linear[2] = scanline[x*ncomp + 2];
                    linear[1] = scanline[x*ncomp + 1];
                    linear[0] = scanline[x*ncomp + 0];
                    break;
            default:
                    linear[0] = linear[1] = linear[2] = scanline[x*ncomp + 0];
                    break;
         }
         stbiw__linear_to_rgbe(rgbe, linear);
         scratch[x + width*0] = rgbe[0];
         scratch[x + width*1] = rgbe[1];
         scratch[x + width*2] = rgbe[2];
         scratch[x + width*3] = rgbe[3];
      }

```

```
      s->func(s->context, scanlineheader, 4);

      /* RLE each component separately */
      for (c=0; c < 4; c++) {
         unsigned char *comp = &scratch[width*c];

         x = 0;
         while (x < width) {
            // find first run
            r = x;
            while (r+2 < width) {
               if (comp[r] == comp[r+1] && comp[r] == comp[r+2])
                  break;
               ++r;
            }
            if (r+2 >= width)
               r = width;
            // dump up to first run
            while (x < r) {
               int len = r-x;
               if (len > 128) len = 128;
               stbiw__write_dump_data(s, len, &comp[x]);
               x += len;
            }
            // if there's a run, output it
            if (r+2 < width) { // same test as what we break out of in search loop, so only true if we break'd
               // find next byte after run
               while (r < width && comp[r] == comp[x])
                  ++r;
               // output run up to r
               while (x < r) {
                  int len = r-x;
                  if (len > 127) len = 127;
                  stbiw__write_run_data(s, len, comp[x]);
                  x += len;
               }
            }
         }
      }
   }
}

static int stbi_write_hdr_core(stbi__write_context *s, int x, int y, int comp, float *data)
{
   if (y <= 0 || x <= 0 || data == NULL)
      return 0;
   else {
      // Each component is stored separately. Allocate scratch space for full output scanline.
      unsigned char *scratch = (unsigned char *) STBIW_MALLOC(x*4);
      int i, len;
      char buffer[128];
      char header[] = "#?RADIANCE\n# Written by stb_image_write.h\nFORMAT=32-bit_rle_rgbe\n";
      s->func(s->context, header, sizeof(header)-1);

#ifdef __STDC_LIB_EXT1__
      len = sprintf_s(buffer, sizeof(buffer), "EXPOSURE=          1.0000000000000\n\n-Y %d +X %d\n", y, x);
#else
      len = sprintf(buffer, "EXPOSURE=          1.0000000000000\n\n-Y %d +X %d\n", y, x);
#endif
      s->func(s->context, buffer, len);

      for(i=0; i < y; i++)
         stbiw__write_hdr_scanline(s, x, comp, scratch, data + comp*x*(stbi__flip_vertically_on_write ? y-1-i : i));
      STBIW_FREE(scratch);
      return 1;
   }
}

STBIWDEF int stbi_write_hdr_to_func(stbi_write_func *func, void *context, int x, int y, int comp, const float *data)
{
   stbi__write_context s = { 0 };
   stbi__start_write_callbacks(&s, func, context);
   return stbi_write_hdr_core(&s, x, y, comp, (float *) data);
}

STBIWDEF int stbi_write_hdr(char const *filename, int x, int y, int comp, const float *data)
{
   stbi__write_context s = { 0 };
   if (stbi__start_write_file(&s,filename)) {
      int r = stbi_write_hdr_core(&s, x, y, comp, (float *) data);
      stbi__end_write_file(&s);
      return r;
   } else
```

```
      return 0;
}
#endif // STBI_WRITE_NO_STDIO


//
// PNG writer
//

#ifndef STBIW_ZLIB_COMPRESS
// stretchy buffer; stbiw__sbpush() == vector<>::push_back() -- stbiw__sbcount() == vector<>::size()
#define stbiw__sbraw(a) ((int *) (void *) (a) - 2)
#define stbiw__sbm(a)   stbiw__sbraw(a)[0]
#define stbiw__sbn(a)   stbiw__sbraw(a)[1]

#define stbiw__sbneedgrow(a,n)  ((a)==0 || stbiw__sbn(a)+n >= stbiw__sbm(a))
#define stbiw__sbmaybegrow(a,n) (stbiw__sbneedgrow(a,(n)) ? stbiw__sbgrow(a,n) : 0)
#define stbiw__sbgrow(a,n)  stbiw__sbgrowf((void **) &(a), (n), sizeof(*(a)))

#define stbiw__sbpush(a, v)      (stbiw__sbmaybegrow(a,1), (a)[stbiw__sbn(a)++] = (v))
#define stbiw__sbcount(a)        ((a) ? stbiw__sbn(a) : 0)
#define stbiw__sbfree(a)         ((a) ? STBIW_FREE(stbiw__sbraw(a)),0 : 0)

static void *stbiw__sbgrowf(void **arr, int increment, int itemsize)
{
   int m = *arr ? 2*stbiw__sbm(*arr)+increment : increment+1;
   void *p = STBIW_REALLOC_SIZED(*arr ? stbiw__sbraw(*arr) : 0, *arr ? (stbiw__sbm(*arr)*itemsize +
      sizeof(int)*2) : 0, itemsize * m + sizeof(int)*2);
   STBIW_ASSERT(p);
   if (p) {
      if (!*arr) ((int *) p)[1] = 0;
      *arr = (void *) ((int *) p + 2);
      stbiw__sbm(*arr) = m;
   }
   return *arr;
}

static unsigned char *stbiw__zlib_flushf(unsigned char *data, unsigned int *bitbuffer, int *bitcount)
{
   while (*bitcount >= 8) {
      stbiw__sbpush(data, STBIW_UCHAR(*bitbuffer));
      *bitbuffer >>= 8;
      *bitcount -= 8;
   }
   return data;
}

static int stbiw__zlib_bitrev(int code, int codebits)
{
   int res=0;
   while (codebits--) {
      res = (res << 1) | (code & 1);
      code >>= 1;
   }
   return res;
}

static unsigned int stbiw__zlib_countm(unsigned char *a, unsigned char *b, int limit)
{
   int i;
   for (i=0; i < limit && i < 258; ++i)
      if (a[i] != b[i]) break;
   return i;
}

static unsigned int stbiw__zhash(unsigned char *data)
{
   stbiw_uint32 hash = data[0] + (data[1] << 8) + (data[2] << 16);
   hash ^= hash << 3;
   hash += hash >> 5;
   hash ^= hash << 4;
   hash += hash >> 17;
   hash ^= hash << 25;
   hash += hash >> 6;
   return hash;
}

#define stbiw__zlib_flush() (out = stbiw__zlib_flushf(out, &bitbuf, &bitcount))
#define stbiw__zlib_add(code,codebits) \
      (bitbuf |= (code) << bitcount, bitcount += (codebits), stbiw__zlib_flush())
#define stbiw__zlib_huffa(b,c)  stbiw__zlib_add(stbiw__zlib_bitrev(b,c),c)
// default huffman tables
#define stbiw__zlib_huff1(n)  stbiw__zlib_huffa(0x30 + (n), 8)
#define stbiw__zlib_huff2(n)  stbiw__zlib_huffa(0x190 + (n)-144, 9)
#define stbiw__zlib_huff3(n)  stbiw__zlib_huffa(0 + (n)-256,7)
#define stbiw__zlib_huff4(n)  stbiw__zlib_huffa(0xc0 + (n)-280,8)
#define stbiw__zlib_huff(n)  ((n) <= 143 ? stbiw__zlib_huff1(n) : (n) <= 255 ? stbiw__zlib_huff2(n) :
```

```
(n) <= 279 ? stbiw__zlib_huff3(n) : stbiw__zlib_huff4(n))
#define stbiw__zlib_huffb(n) ((n) <= 143 ? stbiw__zlib_huff1(n) : stbiw__zlib_huff2(n))

#define stbiw__ZHASH   16384

#endif // STBIW_ZLIB_COMPRESS

STBIWDEF unsigned char * stbi_zlib_compress(unsigned char *data, int data_len, int *out_len, int quality)
{
#ifdef STBIW_ZLIB_COMPRESS
   // user provided a zlib compress implementation, use that
   return STBIW_ZLIB_COMPRESS(data, data_len, out_len, quality);
#else // use builtin
   static unsigned short lengthc[] = { 3,4,5,6,7,8,9,10,11,13,15,17,19,23,27,31,35,43,51,59,67,83,99,115,131,163,195,227,258, 259 };
   static unsigned char  lengtheb[]= { 0,0,0,0,0,0,0, 0, 1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4, 4, 4,  5,  5,  5,  5,  0 };
   static unsigned short distc[]   = { 1,2,3,4,5,7,9,13,17,25,33,49,65,97,129,193,257,385,513,769,1025,1537,2049,3073,4097,6145,8193,12289,16385,24577, 32768 };
   static unsigned char  disteb[]  = { 0,0,0,0,1,1,2,2,3,3,4,4,5,5,6,6,7,7,8,8,9,9,10,10,11,11,12,12,13,13 };
   unsigned int bitbuf=0;
   int i,j, bitcount=0;
   unsigned char *out = NULL;
   unsigned char ***hash_table = (unsigned char***) STBIW_MALLOC(stbiw__ZHASH * sizeof(unsigned char**));
   if (hash_table == NULL)
      return NULL;
   if (quality < 5) quality = 5;

   stbiw__sbpush(out, 0x78);   // DEFLATE 32K window
   stbiw__sbpush(out, 0x5e);   // FLEVEL = 1
   stbiw__zlib_add(1,1);  // BFINAL = 1
   stbiw__zlib_add(1,2);  // BTYPE = 1 -- fixed huffman

   for (i=0; i < stbiw__ZHASH; ++i)
      hash_table[i] = NULL;

   i=0;
   while (i < data_len-3) {
      // hash next 3 bytes of data to be compressed
      int h = stbiw__zhash(data+i)&(stbiw__ZHASH-1), best=3;
      unsigned char *bestloc = 0;
      unsigned char **hlist = hash_table[h];
      int n = stbiw__sbcount(hlist);
      for (j=0; j < n; ++j) {
         if (hlist[j]-data > i-32768) { // if entry lies within window
            int d = stbiw__zlib_countm(hlist[j], data+i, data_len-i);
            if (d >= best) { best=d; bestloc=hlist[j]; }
         }
      }
      // when hash table entry is too long, delete half the entries
      if (hash_table[h] && stbiw__sbn(hash_table[h]) == 2*quality) {
         STBIW_MEMMOVE(hash_table[h], hash_table[h]+quality, sizeof(hash_table[h][0])*quality);
         stbiw__sbn(hash_table[h]) = quality;
      }
      stbiw__sbpush(hash_table[h],data+i);

      if (bestloc) {
         // "lazy matching" - check match at *next* byte, and if it's better, do cur byte as literal
         h = stbiw__zhash(data+i+1)&(stbiw__ZHASH-1);
         hlist = hash_table[h];
         n = stbiw__sbcount(hlist);
         for (j=0; j < n; ++j) {
            if (hlist[j]-data > i-32767) {
               int e = stbiw__zlib_countm(hlist[j], data+i+1, data_len-i-1);
               if (e > best) { // if next match is better, bail on current match
                  bestloc = NULL;
                  break;
               }
            }
         }
      }

      if (bestloc) {
         int d = (int) (data+i - bestloc); // distance back
         STBIW_ASSERT(d <= 32767 && best <= 258);
         for (j=0; best > lengthc[j+1]-1; ++j);
         stbiw__zlib_huff(j+257);
         if (lengtheb[j]) stbiw__zlib_add(best - lengthc[j], lengtheb[j]);
         for (j=0; d > distc[j+1]-1; ++j);
         stbiw__zlib_add(stbiw__zlib_bitrev(j,5),5);
         if (disteb[j]) stbiw__zlib_add(d - distc[j], disteb[j]);
         i += best;
      } else {
```

```
         stbiw__zlib_huffb(data[i]);
         ++i;
      }
   }
   // write out final bytes
   for (;i < data_len; ++i)
      stbiw__zlib_huffb(data[i]);
   stbiw__zlib_huff(256); // end of block
   // pad with 0 bits to byte boundary
   while (bitcount)
      stbiw__zlib_add(0,1);

   for (i=0; i < stbiw__ZHASH; ++i)
      (void) stbiw__sbfree(hash_table[i]);
   STBIW_FREE(hash_table);

   // store uncompressed instead if compression was worse
   if (stbiw__sbn(out) > data_len + 2 + ((data_len+32766)/32767)*5) {
      stbiw__sbn(out) = 2;  // truncate to DEFLATE 32K window and FLEVEL = 1
      for (j = 0; j < data_len;) {
         int blocklen = data_len - j;
         if (blocklen > 32767) blocklen = 32767;
         stbiw__sbpush(out, data_len - j == blocklen); // BFINAL = ?, BTYPE = 0 -- no compression
         stbiw__sbpush(out, STBIW_UCHAR(blocklen)); // LEN
         stbiw__sbpush(out, STBIW_UCHAR(blocklen » 8));
         stbiw__sbpush(out, STBIW_UCHAR(~blocklen)); // NLEN
         stbiw__sbpush(out, STBIW_UCHAR(~blocklen » 8));
         memcpy(out+stbiw__sbn(out), data+j, blocklen);
         stbiw__sbn(out) += blocklen;
         j += blocklen;
      }
   }

   {
      // compute adler32 on input
      unsigned int s1=1, s2=0;
      int blocklen = (int) (data_len % 5552);
      j=0;
      while (j < data_len) {
         for (i=0; i < blocklen; ++i) { s1 += data[j+i]; s2 += s1; }
         s1 %= 65521; s2 %= 65521;
         j += blocklen;
         blocklen = 5552;
      }
      stbiw__sbpush(out, STBIW_UCHAR(s2 » 8));
      stbiw__sbpush(out, STBIW_UCHAR(s2));
      stbiw__sbpush(out, STBIW_UCHAR(s1 » 8));
      stbiw__sbpush(out, STBIW_UCHAR(s1));
   }
   *out_len = stbiw__sbn(out);
   // make returned pointer freeable
   STBIW_MEMMOVE(stbiw__sbraw(out), out, *out_len);
   return (unsigned char *) stbiw__sbraw(out);
#endif // STBIW_ZLIB_COMPRESS
}

static unsigned int stbiw__crc32(unsigned char *buffer, int len)
{
#ifdef STBIW_CRC32
    return STBIW_CRC32(buffer, len);
#else
   static unsigned int crc_table[256] =
   {
      0x00000000, 0x77073096, 0xEE0E612C, 0x990951BA, 0x076DC419, 0x706AF48F, 0xE963A535, 0x9E6495A3,
      0x0eDB8832, 0x79DCB8A4, 0xE0D5E91E, 0x97D2D988, 0x09B64C2B, 0x7EB17CBD, 0xE7B82D07, 0x90BF1D91,
      0x1DB71064, 0x6AB020F2, 0xF3B97148, 0x84BE41DE, 0x1ADAD47D, 0x6DDDE4EB, 0xF4D4B551, 0x83D385C7,
      0x136C9856, 0x646BA8C0, 0xFD62F97A, 0x8A65C9EC, 0x14015C4F, 0x63066CD9, 0xFA0F3D63, 0x8D080DF5,
      0x3B6E20C8, 0x4C69105E, 0xD56041E4, 0xA2677172, 0x3C03E4D1, 0x4B04D447, 0xD20D85FD, 0xA50AB56B,
      0x35B5A8FA, 0x42B2986C, 0xDBBBC9D6, 0xACBCF940, 0x32D86CE3, 0x45DF5C75, 0xDCD60DCF, 0xABD13D59,
      0x26D930AC, 0x51DE003A, 0xC8D75180, 0xBFD06116, 0x21B4F4B5, 0x56B3C423, 0xCFBA9599, 0xB8BDA50F,
      0x2802B89E, 0x5F058808, 0xC60CD9B2, 0xB10BE924, 0x2F6F7C87, 0x58684C11, 0xC1611DAB, 0xB6662D3D,
      0x76DC4190, 0x01DB7106, 0x98D220BC, 0xEFD5102A, 0x71B18589, 0x06B6B51F, 0x9FBFE4A5, 0xE8B8D433,
      0x7807C9A2, 0x0F00F934, 0x9609A88E, 0xE10E9818, 0x7F6A0DBB, 0x086D3D2D, 0x91646C97, 0xE6635C01,
      0x6B6B51F4, 0x1C6C6162, 0x856530D8, 0xF262004E, 0x6C0695ED, 0x1B01A57B, 0x8208F4C1, 0xF50FC457,
      0x65B0D9C6, 0x12B7E950, 0x8BBEB8EA, 0xFCB9887C, 0x62DD1DDF, 0x15DA2D49, 0x8CD37CF3, 0xFBD44C65,
      0x4DB26158, 0x3AB551CE, 0xA3BC0074, 0xD4BB30E2, 0x4ADFA541, 0x3DD895D7, 0xA4D1C46D, 0xD3D6F4FB,
      0x4369E96A, 0x346ED9FC, 0xAD678846, 0xDA60B8D0, 0x44042D73, 0x33031DE5, 0xAA0A4C5F, 0xDD0D7CC9,
      0x5005713C, 0x270241AA, 0xBE0B1010, 0xC90C2086, 0x5768B525, 0x206F85B3, 0xB966D409, 0xCE61E49F,
      0x5EDEF90E, 0x29D9C998, 0xB0D09822, 0xC7D7A8B4, 0x59B33D17, 0x2EB40D81, 0xB7BD5C3B, 0xC0BA6CAD,
      0xEDB88320, 0x9ABFB3B6, 0x03B6E20C, 0x74B1D29A, 0xEAD54739, 0x9DD277AF, 0x04DB2615, 0x73DC1683,
      0xE3630B12, 0x94643B84, 0x0D6D6A3E, 0x7A6A5AA8, 0xE40ECF0B, 0x9309FF9D, 0x0A00AE27, 0x7D079EB1,
      0xF00F9344, 0x8708A3D2, 0x1E01F268, 0x6906C2FE, 0xF762575D, 0x806567CB, 0x196C3671, 0x6E6B06E7,
      0xFED41B76, 0x89D32BE0, 0x10DA7A5A, 0x67DD4ACC, 0xF9B9DF6F, 0x8EBEEFF9, 0x17B7BE43, 0x60B08ED5,
      0xD6D6A3E8, 0xA1D1937E, 0x38D8C2C4, 0x4FDFF252, 0xD1BB67F1, 0xA6BC5767, 0x3FB506DD, 0x48B2364B,
      0xD80D2BDA, 0xAF0A1B4C, 0x36034AF6, 0x41047A60, 0xDF60EFC3, 0xA867DF55, 0x316E8EEF, 0x4669BE79,
      0xCB61B38C, 0xBC66831A, 0x256FD2A0, 0x5268E236, 0xCC0C7795, 0xBB0B4703, 0x220216B9, 0x5505262F,
      0xC5BA3BBE, 0xB2BD0B28, 0x2BB45A92, 0x5CB36A04, 0xC2D7FFA7, 0xB5D0CF31, 0x2CD99E8B, 0x5BDEAE1D,
```

```
      0x9B64C2B0, 0xEC63F226, 0x756AA39C, 0x026D930A, 0x9C0906A9, 0xEB0E363F, 0x72076785, 0x05005713,
      0x95BF4A82, 0xE2B87A14, 0x7BB12BAE, 0x0CB61B38, 0x92D28E9B, 0xE5D5BE0D, 0x7CDCEFB7, 0x0BDBDF21,
      0x86D3D2D4, 0xF1D4E242, 0x68DDB3F8, 0x1FDA836E, 0x81BE16CD, 0xF6B9265B, 0x6FB077E1, 0x18B74777,
      0x88085AE6, 0xFF0F6A70, 0x66063BCA, 0x11010B5C, 0x8F659EFF, 0xF862AE69, 0x616BFFD3, 0x166CCF45,
      0xA00AE278, 0xD70DD2EE, 0x4E048354, 0x3903B3C2, 0xA7672661, 0xD06016F7, 0x4969474D, 0x3E6E77DB,
      0xAED16A4A, 0xD9D65ADC, 0x40DF0B66, 0x37D83BF0, 0xA9BCAE53, 0xDEBB9EC5, 0x47B2CF7F, 0x30B5FFE9,
      0xBDBDF21C, 0xCABAC28A, 0x53B39330, 0x24B4A3A6, 0xBAD03605, 0xCDD70693, 0x54DE5729, 0x23D967BF,
      0xB3667A2E, 0xC4614AB8, 0x5D681B02, 0x2A6F2B94, 0xB40BBE37, 0xC30C8EA1, 0x5A05DF1B, 0x2D02EF8D
   };

   unsigned int crc = ~0u;
   int i;
   for (i=0; i < len; ++i)
      crc = (crc » 8) ^ crc_table[buffer[i] ^ (crc & 0xff)];
   return ~crc;
#endif
}

#define stbiw__wpng4(o,a,b,c,d) ((o)[0]=STBIW_UCHAR(a),(o)[1]=STBIW_UCHAR(b),(o)[2]=STBIW_UCHAR(c),(o)[3]=STBIW_UCHAR(d),(o)+=4)
#define stbiw__wp32(data,v) stbiw__wpng4(data, (v)»24,(v)»16,(v)»8,(v));
#define stbiw__wptag(data,s) stbiw__wpng4(data, s[0],s[1],s[2],s[3])

static void stbiw__wpcrc(unsigned char **data, int len)
{
   unsigned int crc = stbiw__crc32(*data - len - 4, len+4);
   stbiw__wp32(*data, crc);
}

static unsigned char stbiw__paeth(int a, int b, int c)
{
   int p = a + b - c, pa = abs(p-a), pb = abs(p-b), pc = abs(p-c);
   if (pa <= pb && pa <= pc) return STBIW_UCHAR(a);
   if (pb <= pc) return STBIW_UCHAR(b);
   return STBIW_UCHAR(c);
}

// @OPTIMIZE: provide an option that always forces left-predict or paeth predict
static void stbiw__encode_png_line(unsigned char *pixels, int stride_bytes, int width, int height, int y, int n, int filter_type, signed char *line_buffer)
{
   static int mapping[] = { 0,1,2,3,4 };
   static int firstmap[] = { 0,1,0,5,6 };
   int *mymap = (y != 0) ? mapping : firstmap;
   int i;
   int type = mymap[filter_type];
   unsigned char *z = pixels + stride_bytes * (stbi__flip_vertically_on_write ? height-1-y : y);
   int signed_stride = stbi__flip_vertically_on_write ? -stride_bytes : stride_bytes;

   if (type==0) {
      memcpy(line_buffer, z, width*n);
      return;
   }

   // first loop isn't optimized since it's just one pixel
   for (i = 0; i < n; ++i) {
      switch (type) {
         case 1: line_buffer[i] = z[i]; break;
         case 2: line_buffer[i] = z[i] - z[i-signed_stride]; break;
         case 3: line_buffer[i] = z[i] - (z[i-signed_stride]»1); break;
         case 4: line_buffer[i] = (signed char) (z[i] - stbiw__paeth(0,z[i-signed_stride],0)); break;
         case 5: line_buffer[i] = z[i]; break;
         case 6: line_buffer[i] = z[i]; break;
      }
   }
   switch (type) {
      case 1: for (i=n; i < width*n; ++i) line_buffer[i] = z[i] - z[i-n]; break;
      case 2: for (i=n; i < width*n; ++i) line_buffer[i] = z[i] - z[i-signed_stride]; break;
      case 3: for (i=n; i < width*n; ++i) line_buffer[i] = z[i] - ((z[i-n] + z[i-signed_stride])»1); break;
      case 4: for (i=n; i < width*n; ++i) line_buffer[i] = z[i] - stbiw__paeth(z[i-n], z[i-signed_stride], z[i-signed_stride-n]); break;
      case 5: for (i=n; i < width*n; ++i) line_buffer[i] = z[i] - (z[i-n]»1); break;
      case 6: for (i=n; i < width*n; ++i) line_buffer[i] = z[i] - stbiw__paeth(z[i-n], 0,0); break;
   }
}

STBIWDEF unsigned char *stbi_write_png_to_mem(const unsigned char *pixels, int stride_bytes, int x, int y, int n, int *out_len)
{
   int force_filter = stbi_write_force_png_filter;
   int ctype[5] = { -1, 0, 4, 2, 6 };
   unsigned char sig[8] = { 137,80,78,71,13,10,26,10 };
   unsigned char *out,*o, *filt, *zlib;
   signed char *line_buffer;
   int j,zlen;

```

```
   if (stride_bytes == 0)
      stride_bytes = x * n;

   if (force_filter >= 5) {
      force_filter = -1;
   }

   filt = (unsigned char *) STBIW_MALLOC((x*n+1) * y); if (!filt) return 0;
   line_buffer = (signed char *) STBIW_MALLOC(x * n); if (!line_buffer) { STBIW_FREE(filt); return 0;
}
   for (j=0; j < y; ++j) {
      int filter_type;
      if (force_filter > -1) {
         filter_type = force_filter;
         stbiw__encode_png_line((unsigned char*)(pixels), stride_bytes, x, y, j, n, force_filter,
line_buffer);
      } else { // Estimate the best filter by running through all of them:
         int best_filter = 0, best_filter_val = 0x7fffffff, est, i;
         for (filter_type = 0; filter_type < 5; filter_type++) {
            stbiw__encode_png_line((unsigned char*)(pixels), stride_bytes, x, y, j, n, filter_type,
line_buffer);

            // Estimate the entropy of the line using this filter; the less, the better.
            est = 0;
            for (i = 0; i < x*n; ++i) {
               est += abs((signed char) line_buffer[i]);
            }
            if (est < best_filter_val) {
               best_filter_val = est;
               best_filter = filter_type;
            }
         }
         if (filter_type != best_filter) {  // If the last iteration already got us the best filter,
don't redo it
            stbiw__encode_png_line((unsigned char*)(pixels), stride_bytes, x, y, j, n, best_filter,
line_buffer);
            filter_type = best_filter;
         }
      }
      // when we get here, filter_type contains the filter type, and line_buffer contains the data
      filt[j*(x*n+1)] = (unsigned char) filter_type;
      STBIW_MEMMOVE(filt+j*(x*n+1)+1, line_buffer, x*n);
   }
   STBIW_FREE(line_buffer);
   zlib = stbi_zlib_compress(filt, y*( x*n+1), &zlen, stbi_write_png_compression_level);
   STBIW_FREE(filt);
   if (!zlib) return 0;

   // each tag requires 12 bytes of overhead
   out = (unsigned char *) STBIW_MALLOC(8 + 12+13 + 12+zlen + 12);
   if (!out) return 0;
   *out_len = 8 + 12+13 + 12+zlen + 12;

   o=out;
   STBIW_MEMMOVE(o,sig,8); o+= 8;
   stbiw__wp32(o, 13); // header length
   stbiw__wptag(o, "IHDR");
   stbiw__wp32(o, x);
   stbiw__wp32(o, y);
   *o++ = 8;
   *o++ = STBIW_UCHAR(ctype[n]);
   *o++ = 0;
   *o++ = 0;
   *o++ = 0;
   stbiw__wpcrc(&o,13);

   stbiw__wp32(o, zlen);
   stbiw__wptag(o, "IDAT");
   STBIW_MEMMOVE(o, zlib, zlen);
   o += zlen;
   STBIW_FREE(zlib);
   stbiw__wpcrc(&o, zlen);

   stbiw__wp32(o,0);
   stbiw__wptag(o, "IEND");
   stbiw__wpcrc(&o,0);

   STBIW_ASSERT(o == out + *out_len);

   return out;
}

#ifndef STBI_WRITE_NO_STDIO
STBIWDEF int stbi_write_png(char const *filename, int x, int y, int comp, const void *data, int
stride_bytes)
{
   FILE *f;
```

```
   int len;
   unsigned char *png = stbi_write_png_to_mem((const unsigned char *) data, stride_bytes, x, y, comp, &len);
   if (png == NULL) return 0;

   f = stbiw__fopen(filename, "wb");
   if (!f) { STBIW_FREE(png); return 0; }
   fwrite(png, 1, len, f);
   fclose(f);
   STBIW_FREE(png);
   return 1;
}
#endif

STBIWDEF int stbi_write_png_to_func(stbi_write_func *func, void *context, int x, int y, int comp, const void *data, int stride_bytes)
{
   int len;
   unsigned char *png = stbi_write_png_to_mem((const unsigned char *) data, stride_bytes, x, y, comp, &len);
   if (png == NULL) return 0;
   func(context, png, len);
   STBIW_FREE(png);
   return 1;
}


/* ***************************************************************************
 *
 * JPEG writer
 *
 * This is based on Jon Olick's jo_jpeg.cpp:
 * public domain Simple, Minimalistic JPEG writer - http://www.jonolick.com/code.html
 */

static const unsigned char stbiw__jpg_ZigZag[] = {0,1,5,6,14,15,27,28,2,4,7,13,16,26,29,42,3,8,12,17,25,30,41,43,9,11,18,

24,31,40,44,53,10,19,23,32,39,45,52,54,20,22,33,38,46,51,55,60,21,34,37,47,50,56,59,61,35,36,48,49,57,58,62,63};

static void stbiw__jpg_writeBits(stbi__write_context *s, int *bitBufP, int *bitCntP, const unsigned short *bs) {
   int bitBuf = *bitBufP, bitCnt = *bitCntP;
   bitCnt += bs[1];
   bitBuf |= bs[0] << (24 - bitCnt);
   while(bitCnt >= 8) {
      unsigned char c = (bitBuf >> 16) & 255;
      stbiw__putc(s, c);
      if(c == 255) {
         stbiw__putc(s, 0);
      }
      bitBuf <<= 8;
      bitCnt -= 8;
   }
   *bitBufP = bitBuf;
   *bitCntP = bitCnt;
}

static void stbiw__jpg_DCT(float *d0p, float *d1p, float *d2p, float *d3p, float *d4p, float *d5p, float *d6p, float *d7p) {
   float d0 = *d0p, d1 = *d1p, d2 = *d2p, d3 = *d3p, d4 = *d4p, d5 = *d5p, d6 = *d6p, d7 = *d7p;
   float z1, z2, z3, z4, z5, z11, z13;

   float tmp0 = d0 + d7;
   float tmp7 = d0 - d7;
   float tmp1 = d1 + d6;
   float tmp6 = d1 - d6;
   float tmp2 = d2 + d5;
   float tmp5 = d2 - d5;
   float tmp3 = d3 + d4;
   float tmp4 = d3 - d4;

   // Even part
   float tmp10 = tmp0 + tmp3;   // phase 2
   float tmp13 = tmp0 - tmp3;
   float tmp11 = tmp1 + tmp2;
   float tmp12 = tmp1 - tmp2;

   d0 = tmp10 + tmp11;       // phase 3
   d4 = tmp10 - tmp11;

   z1 = (tmp12 + tmp13) * 0.707106781f; // c4
   d2 = tmp13 + z1;       // phase 5
   d6 = tmp13 - z1;

   // Odd part
```

```
   tmp10 = tmp4 + tmp5;       // phase 2
   tmp11 = tmp5 + tmp6;
   tmp12 = tmp6 + tmp7;

   // The rotator is modified from fig 4-8 to avoid extra negations.
   z5 = (tmp10 - tmp12) * 0.382683433f; // c6
   z2 = tmp10 * 0.541196100f + z5; // c2-c6
   z4 = tmp12 * 1.306562965f + z5; // c2+c6
   z3 = tmp11 * 0.707106781f; // c4

   z11 = tmp7 + z3;      // phase 5
   z13 = tmp7 - z3;

   *d5p = z13 + z2;         // phase 6
   *d3p = z13 - z2;
   *d1p = z11 + z4;
   *d7p = z11 - z4;

   *d0p = d0;  *d2p = d2;  *d4p = d4;  *d6p = d6;
}

static void stbiw__jpg_calcBits(int val, unsigned short bits[2]) {
   int tmp1 = val < 0 ? -val : val;
   val = val < 0 ? val-1 : val;
   bits[1] = 1;
   while(tmp1 >> 1) {
      ++bits[1];
   }
   bits[0] = val & ((1<<bits[1])-1);
}

static int stbiw__jpg_processDU(stbi__write_context *s, int *bitBuf, int *bitCnt, float *CDU, int du_stride, float *fdtbl, int DC, const unsigned short HTDC[256][2], const unsigned short HTAC[256][2]) {
   const unsigned short EOB[2] = { HTAC[0x00][0], HTAC[0x00][1] };
   const unsigned short M16zeroes[2] = { HTAC[0xF0][0], HTAC[0xF0][1] };
   int dataOff, i, j, n, diff, end0pos, x, y;
   int DU[64];

   // DCT rows
   for(dataOff=0, n=du_stride*8; dataOff<n; dataOff+=du_stride) {
      stbiw__jpg_DCT(&CDU[dataOff], &CDU[dataOff+1], &CDU[dataOff+2], &CDU[dataOff+3], &CDU[dataOff+4], &CDU[dataOff+5], &CDU[dataOff+6], &CDU[dataOff+7]);
   }
   // DCT columns
   for(dataOff=0; dataOff<8; ++dataOff) {
      stbiw__jpg_DCT(&CDU[dataOff], &CDU[dataOff+du_stride], &CDU[dataOff+du_stride*2], &CDU[dataOff+du_stride*3], &CDU[dataOff+du_stride*4],
                     &CDU[dataOff+du_stride*5], &CDU[dataOff+du_stride*6], &CDU[dataOff+du_stride*7]);
   }
   // Quantize/descale/zigzag the coefficients
   for(y = 0, j=0; y < 8; ++y) {
      for(x = 0; x < 8; ++x,++j) {
         float v;
         i = y*du_stride+x;
         v = CDU[i]*fdtbl[j];
         // DU[stbiw__jpg_ZigZag[j]] = (int)(v < 0 ? ceilf(v - 0.5f) : floorf(v + 0.5f));
         // ceilf() and floorf() are C99, not C89, but I /think/ they're not needed here anyway?
         DU[stbiw__jpg_ZigZag[j]] = (int)(v < 0 ? v - 0.5f : v + 0.5f);
      }
   }

   // Encode DC
   diff = DU[0] - DC;
   if (diff == 0) {
      stbiw__jpg_writeBits(s, bitBuf, bitCnt, HTDC[0]);
   } else {
      unsigned short bits[2];
      stbiw__jpg_calcBits(diff, bits);
      stbiw__jpg_writeBits(s, bitBuf, bitCnt, HTDC[bits[1]]);
      stbiw__jpg_writeBits(s, bitBuf, bitCnt, bits);
   }
   // Encode ACs
   end0pos = 63;
   for(; (end0pos>0)&&(DU[end0pos]==0); --end0pos) {
   }
   // end0pos = first element in reverse order !=0
   if(end0pos == 0) {
      stbiw__jpg_writeBits(s, bitBuf, bitCnt, EOB);
      return DU[0];
   }
   for(i = 1; i <= end0pos; ++i) {
      int startpos = i;
      int nrzeroes;
      unsigned short bits[2];
      for (; DU[i]==0 && i<=end0pos; ++i) {
      }
```

```
      nrzeroes = i-startpos;
      if ( nrzeroes >= 16 ) {
         int lng = nrzeroes»4;
         int nrmarker;
         for (nrmarker=1; nrmarker <= lng; ++nrmarker)
            stbiw__jpg_writeBits(s, bitBuf, bitCnt, M16zeroes);
         nrzeroes &= 15;
      }
      stbiw__jpg_calcBits(DU[i], bits);
      stbiw__jpg_writeBits(s, bitBuf, bitCnt, HTAC[(nrzeroes«4)+bits[1]]);
      stbiw__jpg_writeBits(s, bitBuf, bitCnt, bits);
   }
   if(end0pos != 63) {
      stbiw__jpg_writeBits(s, bitBuf, bitCnt, EOB);
   }
   return DU[0];
}

static int stbi_write_jpg_core(stbi__write_context *s, int width, int height, int comp, const void*
data, int quality) {
   // Constants that don't pollute global namespace
   static const unsigned char std_dc_luminance_nrcodes[] = {0,0,1,5,1,1,1,1,1,1,0,0,0,0,0,0,0};
   static const unsigned char std_dc_luminance_values[] = {0,1,2,3,4,5,6,7,8,9,10,11};
   static const unsigned char std_ac_luminance_nrcodes[] = {0,0,2,1,3,3,2,4,3,5,5,4,4,0,0,1,0x7d};
   static const unsigned char std_ac_luminance_values[] = {

0x01,0x02,0x03,0x00,0x04,0x11,0x05,0x12,0x21,0x31,0x41,0x06,0x13,0x51,0x61,0x07,0x22,0x71,0x14,0x32,0x81,0x91,0xa1,0x08,

0x23,0x42,0xb1,0xc1,0x15,0x52,0xd1,0xf0,0x24,0x33,0x62,0x72,0x82,0x09,0x0a,0x16,0x17,0x18,0x19,0x1a,0x25,0x26,0x27,0x28,

0x29,0x2a,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x53,0x54,0x55,0x56,0x57,0x58,0x59,

0x5a,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x83,0x84,0x85,0x86,0x87,0x88,0x89,

0x8a,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xb2,0xb3,0xb4,0xb5,0xb6,

0xb7,0xb8,0xb9,0xba,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xe1,0xe2,
      0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa
   };
   static const unsigned char std_dc_chrominance_nrcodes[] = {0,0,3,1,1,1,1,1,1,1,1,1,0,0,0,0,0};
   static const unsigned char std_dc_chrominance_values[] = {0,1,2,3,4,5,6,7,8,9,10,11};
   static const unsigned char std_ac_chrominance_nrcodes[] = {0,0,2,1,2,4,4,3,4,7,5,4,4,0,1,2,0x77};
   static const unsigned char std_ac_chrominance_values[] = {

0x00,0x01,0x02,0x03,0x11,0x04,0x05,0x21,0x31,0x06,0x12,0x41,0x51,0x07,0x61,0x71,0x13,0x22,0x32,0x81,0x08,0x14,0x42,0x91,

0xa1,0xb1,0xc1,0x09,0x23,0x33,0x52,0xf0,0x15,0x62,0x72,0xd1,0x0a,0x16,0x24,0x34,0xe1,0x25,0xf1,0x17,0x18,0x19,0x1a,0x26,

0x27,0x28,0x29,0x2a,0x35,0x36,0x37,0x38,0x39,0x3a,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x53,0x54,0x55,0x56,0x57,0x58,

0x59,0x5a,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x82,0x83,0x84,0x85,0x86,0x87,

0x88,0x89,0x8a,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xb2,0xb3,0xb4,

0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,
      0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa
   };
   // Huffman tables
   static const unsigned short YDC_HT[256][2] = {
{0,2},{2,3},{3,3},{4,3},{5,3},{6,3},{14,4},{30,5},{62,6},{126,7},{254,8},{510,9}};
   static const unsigned short UVDC_HT[256][2] = {
{0,2},{1,2},{2,2},{6,3},{14,4},{30,5},{62,6},{126,7},{254,8},{510,9},{1022,10},{2046,11}};
   static const unsigned short YAC_HT[256][2] = {

{10,4},{0,2},{1,2},{4,3},{11,4},{26,5},{120,7},{248,8},{1014,10},{65410,16},{65411,16},{0,0},{0,0},{0,0},{0,0},{0,0},{0,

{12,4},{27,5},{121,7},{502,9},{2038,11},{65412,16},{65413,16},{65414,16},{65415,16},{65416,16},{0,0},{0,0},{0,0},{0,0},

{28,5},{249,8},{1015,10},{4084,12},{65417,16},{65418,16},{65419,16},{65420,16},{65421,16},{65422,16},{0,0},{0,0},{0,0},

{58,6},{503,9},{4085,12},{65423,16},{65424,16},{65425,16},{65426,16},{65427,16},{65428,16},{65429,16},{0,0},{0,0},{0,0},

{59,6},{1016,10},{65430,16},{65431,16},{65432,16},{65433,16},{65434,16},{65435,16},{65436,16},{65437,16},{0,0},{0,0},{0,

{122,7},{2039,11},{65438,16},{65439,16},{65440,16},{65441,16},{65442,16},{65443,16},{65444,16},{65445,16},{0,0},{0,0},{

{123,7},{4086,12},{65446,16},{65447,16},{65448,16},{65449,16},{65450,16},{65451,16},{65452,16},{65453,16},{0,0},{0,0},{

{250,8},{4087,12},{65454,16},{65455,16},{65456,16},{65457,16},{65458,16},{65459,16},{65460,16},{65461,16},{0,0},{0,0},{

{504,9},{32704,15},{65462,16},{65463,16},{65464,16},{65465,16},{65466,16},{65467,16},{65468,16},{65469,16},{0,0},{0,0},

{505,9},{65470,16},{65471,16},{65472,16},{65473,16},{65474,16},{65475,16},{65476,16},{65477,16},{65478,16},{0,0},{0,0},

{506,9},{65479,16},{65480,16},{65481,16},{65482,16},{65483,16},{65484,16},{65485,16},{65486,16},{65487,16},{0,0},{0,0},

{1017,10},{65488,16},{65489,16},{65490,16},{65491,16},{65492,16},{65493,16},{65494,16},{65495,16},{65496,16},{0,0},{0,0
```

```
{1018,10},{65497,16},{65498,16},{65499,16},{65500,16},{65501,16},{65502,16},{65503,16},{65504,16},{65505,16},{0,0},{0,0
{2040,11},{65506,16},{65507,16},{65508,16},{65509,16},{65510,16},{65511,16},{65512,16},{65513,16},{65514,16},{0,0},{0,0
{65515,16},{65516,16},{65517,16},{65518,16},{65519,16},{65520,16},{65521,16},{65522,16},{65523,16},{65524,16},{0,0},{0,0
{2041,11},{65525,16},{65526,16},{65527,16},{65528,16},{65529,16},{65530,16},{65531,16},{65532,16},{65533,16},{65534,16},
   };
   static const unsigned short UVAC_HT[256][2] = {
{0,2},{1,2},{4,3},{10,4},{24,5},{25,5},{56,6},{120,7},{500,9},{1014,10},{4084,12},{0,0},{0,0},{0,0},{0,0},{0,0},{0,0},
{11,4},{57,6},{246,8},{501,9},{2038,11},{4085,12},{65416,16},{65417,16},{65418,16},{65419,16},{0,0},{0,0},{0,0},{0,0},
{26,5},{247,8},{1015,10},{4086,12},{32706,15},{65420,16},{65421,16},{65422,16},{65423,16},{65424,16},{0,0},{0,0},{0,0},
{27,5},{248,8},{1016,10},{4087,12},{65425,16},{65426,16},{65427,16},{65428,16},{65429,16},{65430,16},{0,0},{0,0},{0,0},
{58,6},{502,9},{65431,16},{65432,16},{65433,16},{65434,16},{65435,16},{65436,16},{65437,16},{65438,16},{0,0},{0,0},{0,0
{59,6},{1017,10},{65439,16},{65440,16},{65441,16},{65442,16},{65443,16},{65444,16},{65445,16},{65446,16},{0,0},{0,0},{0,
{121,7},{2039,11},{65447,16},{65448,16},{65449,16},{65450,16},{65451,16},{65452,16},{65453,16},{65454,16},{0,0},{0,0},{
{122,7},{2040,11},{65455,16},{65456,16},{65457,16},{65458,16},{65459,16},{65460,16},{65461,16},{65462,16},{0,0},{0,0},{
{249,8},{65463,16},{65464,16},{65465,16},{65466,16},{65467,16},{65468,16},{65469,16},{65470,16},{65471,16},{0,0},{0,0},
{503,9},{65472,16},{65473,16},{65474,16},{65475,16},{65476,16},{65477,16},{65478,16},{65479,16},{65480,16},{0,0},{0,0},
{504,9},{65481,16},{65482,16},{65483,16},{65484,16},{65485,16},{65486,16},{65487,16},{65488,16},{65489,16},{0,0},{0,0},
{505,9},{65490,16},{65491,16},{65492,16},{65493,16},{65494,16},{65495,16},{65496,16},{65497,16},{65498,16},{0,0},{0,0},
{506,9},{65499,16},{65500,16},{65501,16},{65502,16},{65503,16},{65504,16},{65505,16},{65506,16},{65507,16},{0,0},{0,0},
{2041,11},{65508,16},{65509,16},{65510,16},{65511,16},{65512,16},{65513,16},{65514,16},{65515,16},{65516,16},{0,0},{0,0
{16352,14},{65517,16},{65518,16},{65519,16},{65520,16},{65521,16},{65522,16},{65523,16},{65524,16},{65525,16},{0,0},{0,
{1018,10},{32707,15},{65526,16},{65527,16},{65528,16},{65529,16},{65530,16},{65531,16},{65532,16},{65533,16},{65534,16},
   };
   static const int YQT[] =
{16,11,10,16,24,40,51,61,12,12,14,19,26,58,60,55,14,13,16,24,40,57,69,56,14,17,22,29,51,87,80,62,18,22,
37,56,68,109,103,77,24,35,55,64,81,104,113,92,49,64,78,87,103,121,120,101,72,92,95,98,112,100,103,99};
   static const int UVQT[] =
{17,18,24,47,99,99,99,99,18,21,26,66,99,99,99,99,24,26,56,99,99,99,99,99,47,66,99,99,99,99,99,99,
99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99,99};
   static const float aasf[] = { 1.0f * 2.828427125f, 1.387039845f * 2.828427125f, 1.306562965f *
2.828427125f, 1.175875602f * 2.828427125f,
                                 1.0f * 2.828427125f, 0.785694958f * 2.828427125f, 0.541196100f *
2.828427125f, 0.275899379f * 2.828427125f };

   int row, col, i, k, subsample;
   float fdtbl_Y[64], fdtbl_UV[64];
   unsigned char YTable[64], UVTable[64];

   if(!data || !width || !height || comp > 4 || comp < 1) {
      return 0;
   }

   quality = quality ? quality : 90;
   subsample = quality <= 90 ? 1 : 0;
   quality = quality < 1 ? 1 : quality > 100 ? 100 : quality;
   quality = quality < 50 ? 5000 / quality : 200 - quality * 2;

   for(i = 0; i < 64; ++i) {
      int uvti, yti = (YQT[i]*quality+50)/100;
      YTable[stbiw__jpg_ZigZag[i]] = (unsigned char) (yti < 1 ? 1 : yti > 255 ? 255 : yti);
      uvti = (UVQT[i]*quality+50)/100;
      UVTable[stbiw__jpg_ZigZag[i]] = (unsigned char) (uvti < 1 ? 1 : uvti > 255 ? 255 : uvti);
   }

   for(row = 0, k = 0; row < 8; ++row) {
      for(col = 0; col < 8; ++col, ++k) {
         fdtbl_Y[k]  = 1 / (YTable [stbiw__jpg_ZigZag[k]] * aasf[row] * aasf[col]);
         fdtbl_UV[k] = 1 / (UVTable[stbiw__jpg_ZigZag[k]] * aasf[row] * aasf[col]);
      }
   }

   // Write Headers
   {
      static const unsigned char head0[] = {
0xFF,0xD8,0xFF,0xE0,0,0x10,'J','F','I','F',0,1,1,0,0,1,0,1,0,0,0xFF,0xDB,0,0x84,0 };
```

```
      static const unsigned char head2[] = { 0xFF,0xDA,0,0xC,3,1,0,2,0x11,3,0x11,0,0x3F,0 };
      const unsigned char head1[] = { 0xFF,0xC0,0,0x11,8,(unsigned char)(height»8),STBIW_UCHAR(height),(unsigned char)(width»8),STBIW_UCHAR(width),
                                      3,1,(unsigned char)(subsample?0x22:0x11),0,2,0x11,1,3,0x11,1,0xFF,0xC4,0x01,0xA2,0 };
      s->func(s->context, (void*)head0, sizeof(head0));
      s->func(s->context, (void*)YTable, sizeof(YTable));
      stbiw__putc(s, 1);
      s->func(s->context, UVTable, sizeof(UVTable));
      s->func(s->context, (void*)head1, sizeof(head1));
      s->func(s->context, (void*)(std_dc_luminance_nrcodes+1), sizeof(std_dc_luminance_nrcodes)-1);
      s->func(s->context, (void*)std_dc_luminance_values, sizeof(std_dc_luminance_values));
      stbiw__putc(s, 0x10); // HTYACinfo
      s->func(s->context, (void*)(std_ac_luminance_nrcodes+1), sizeof(std_ac_luminance_nrcodes)-1);
      s->func(s->context, (void*)std_ac_luminance_values, sizeof(std_ac_luminance_values));
      stbiw__putc(s, 1); // HTUDCinfo
      s->func(s->context, (void*)(std_dc_chrominance_nrcodes+1), sizeof(std_dc_chrominance_nrcodes)-1);
      s->func(s->context, (void*)std_dc_chrominance_values, sizeof(std_dc_chrominance_values));
      stbiw__putc(s, 0x11); // HTUACinfo
      s->func(s->context, (void*)(std_ac_chrominance_nrcodes+1), sizeof(std_ac_chrominance_nrcodes)-1);
      s->func(s->context, (void*)std_ac_chrominance_values, sizeof(std_ac_chrominance_values));
      s->func(s->context, (void*)head2, sizeof(head2));
   }

   // Encode 8x8 macroblocks
   {
      static const unsigned short fillBits[] = {0x7F, 7};
      int DCY=0, DCU=0, DCV=0;
      int bitBuf=0, bitCnt=0;
      // comp == 2 is grey+alpha (alpha is ignored)
      int ofsG = comp > 2 ? 1 : 0, ofsB = comp > 2 ? 2 : 0;
      const unsigned char *dataR = (const unsigned char *)data;
      const unsigned char *dataG = dataR + ofsG;
      const unsigned char *dataB = dataR + ofsB;
      int x, y, pos;
      if(subsample) {
         for(y = 0; y < height; y += 16) {
            for(x = 0; x < width; x += 16) {
               float Y[256], U[256], V[256];
               for(row = y, pos = 0; row < y+16; ++row) {
                  // row >= height => use last input row
                  int clamped_row = (row < height) ? row : height - 1;
                  int base_p = (stbi__flip_vertically_on_write ? (height-1-clamped_row) : clamped_row)*width*comp;
                  for(col = x; col < x+16; ++col, ++pos) {
                     // if col >= width => use pixel from last input column
                     int p = base_p + ((col < width) ? col : (width-1))*comp;
                     float r = dataR[p], g = dataG[p], b = dataB[p];
                     Y[pos]= +0.29900f*r + 0.58700f*g + 0.11400f*b - 128;
                     U[pos]= -0.16874f*r - 0.33126f*g + 0.50000f*b;
                     V[pos]= +0.50000f*r - 0.41869f*g - 0.08131f*b;
                  }
               }
               DCY = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, Y+0,   16, fdtbl_Y, DCY, YDC_HT, YAC_HT);
               DCY = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, Y+8,   16, fdtbl_Y, DCY, YDC_HT, YAC_HT);
               DCY = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, Y+128, 16, fdtbl_Y, DCY, YDC_HT, YAC_HT);
               DCY = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, Y+136, 16, fdtbl_Y, DCY, YDC_HT, YAC_HT);

               // subsample U,V
               {
                  float subU[64], subV[64];
                  int yy, xx;
                  for(yy = 0, pos = 0; yy < 8; ++yy) {
                     for(xx = 0; xx < 8; ++xx, ++pos) {
                        int j = yy*32+xx*2;
                        subU[pos] = (U[j+0] + U[j+1] + U[j+16] + U[j+17]) * 0.25f;
                        subV[pos] = (V[j+0] + V[j+1] + V[j+16] + V[j+17]) * 0.25f;
                     }
                  }
                  DCU = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, subU, 8, fdtbl_UV, DCU, UVDC_HT, UVAC_HT);
                  DCV = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, subV, 8, fdtbl_UV, DCV, UVDC_HT, UVAC_HT);
               }
            }
         }
      } else {
         for(y = 0; y < height; y += 8) {
            for(x = 0; x < width; x += 8) {
               float Y[64], U[64], V[64];
               for(row = y, pos = 0; row < y+8; ++row) {
```

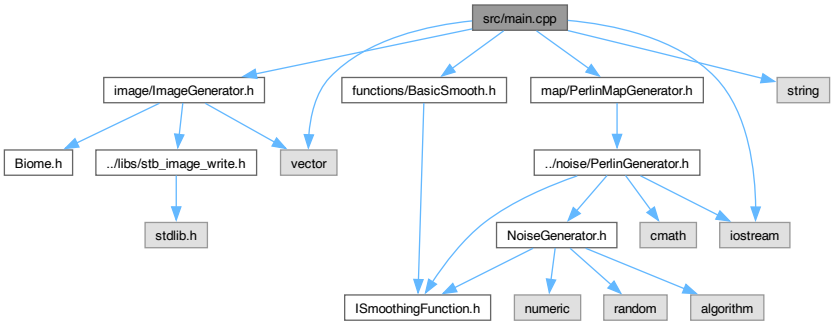
```
                  // row >= height => use last input row
                  int clamped_row = (row < height) ? row : height - 1;
                  int base_p = (stbi__flip_vertically_on_write ? (height-1-clamped_row) : clamped_row)*width*comp;
                  for(col = x; col < x+8; ++col, ++pos) {
                     // if col >= width => use pixel from last input column
                     int p = base_p + ((col < width) ? col : (width-1))*comp;
                     float r = dataR[p], g = dataG[p], b = dataB[p];
                     Y[pos]= +0.29900f*r + 0.58700f*g + 0.11400f*b - 128;
                     U[pos]= -0.16874f*r - 0.33126f*g + 0.50000f*b;
                     V[pos]= +0.50000f*r - 0.41869f*g - 0.08131f*b;
                  }
               }

               DCY = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, Y, 8, fdtbl_Y,  DCY, YDC_HT, YAC_HT);
               DCU = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, U, 8, fdtbl_UV, DCU, UVDC_HT, UVAC_HT);
               DCV = stbiw__jpg_processDU(s, &bitBuf, &bitCnt, V, 8, fdtbl_UV, DCV, UVDC_HT, UVAC_HT);
            }
         }
      }

      // Do the bit alignment of the EOI marker
      stbiw__jpg_writeBits(s, &bitBuf, &bitCnt, fillBits);
   }

   // EOI
   stbiw__putc(s, 0xFF);
   stbiw__putc(s, 0xD9);

   return 1;
}

STBIWDEF int stbi_write_jpg_to_func(stbi_write_func *func, void *context, int x, int y, int comp, const void *data, int quality)
{
   stbi__write_context s = { 0 };
   stbi__start_write_callbacks(&s, func, context);
   return stbi_write_jpg_core(&s, x, y, comp, (void *) data, quality);
}


#ifndef STBI_WRITE_NO_STDIO
STBIWDEF int stbi_write_jpg(char const *filename, int x, int y, int comp, const void *data, int quality)
{
   stbi__write_context s = { 0 };
   if (stbi__start_write_file(&s,filename)) {
      int r = stbi_write_jpg_core(&s, x, y, comp, data, quality);
      stbi__end_write_file(&s);
      return r;
   } else
      return 0;
}
#endif

#endif // STB_IMAGE_WRITE_IMPLEMENTATION

/* Revision history
      1.16  (2021-07-11)
             make Deflate code emit uncompressed blocks when it would otherwise expand
             support writing BMPs with alpha channel
      1.15  (2020-07-13) unknown
      1.14  (2020-02-02) updated JPEG writer to downsample chroma channels
      1.13
      1.12
      1.11  (2019-08-11)

      1.10  (2019-02-07)
             support utf8 filenames in Windows; fix warnings and platform ifdefs
      1.09  (2018-02-11)
             fix typo in zlib quality API, improve STB_I_W_STATIC in C++
      1.08  (2018-01-29)
             add stbi__flip_vertically_on_write, external zlib, zlib quality, choose PNG filter
      1.07  (2017-07-24)
             doc fix
      1.06 (2017-07-23)
             writing JPEG (using Jon Olick's code)
      1.05   ???
      1.04 (2017-03-03)
             monochrome BMP expansion
      1.03   ???
      1.02 (2016-04-02)
             avoid allocating large structures on the stack
      1.01 (2016-01-16)
             STBIW_REALLOC_SIZED: support allocators with no realloc support
             avoid race-condition in crc initialization
             minor compile issues
```

```
      1.00 (2015-09-14)
             installable file IO function
      0.99 (2015-09-13)
             warning fixes; TGA rle support
      0.98 (2015-04-08)
             added STBIW_MALLOC, STBIW_ASSERT etc
      0.97 (2015-01-18)
             fixed HDR asserts, rewrote HDR rle logic
      0.96 (2015-01-17)
             add HDR output
             fix monochrome BMP
      0.95 (2014-08-17)
             add monochrome TGA output
      0.94 (2014-05-31)
             rename private functions to avoid conflicts with stb_image.h
      0.93 (2014-05-27)
             warning fixes
      0.92 (2010-08-01)
             casts to unsigned char to fix warnings
      0.91 (2010-07-17)
             first public release
      0.90   first internal release
*/

/*
------------------------------------------------------------------------------
This software is available under 2 licenses -- choose whichever you prefer.
------------------------------------------------------------------------------
ALTERNATIVE A - MIT License
Copyright (c) 2017 Sean Barrett
Permission is hereby granted, free of charge, to any person obtaining a copy of
this software and associated documentation files (the "Software"), to deal in
the Software without restriction, including without limitation the rights to
use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies
of the Software, and to permit persons to whom the Software is furnished to do
so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
------------------------------------------------------------------------------
ALTERNATIVE B - Public Domain (www.unlicense.org)
This is free and unencumbered software released into the public domain.
Anyone is free to copy, modify, publish, use, compile, sell, or distribute this
software, either in source code form or as a compiled binary, for any purpose,
commercial or non-commercial, and by any means.
In jurisdictions that recognize copyright laws, the author or authors of this
software dedicate any and all copyright interest in the software to the public
domain. We make this dedication for the benefit of the public at large and to
the detriment of our heirs and successors. We intend this dedication to be an
overt act of relinquishment in perpetuity of all present and future rights to
this software under copyright law.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN
ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
------------------------------------------------------------------------------
*/
```

7.12 src/main.cpp File Reference

```
#include "image/ImageGenerator.h"
#include "functions/BasicSmooth.h"
#include "map/PerlinMapGenerator.h"
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for main.cpp:



Functions

- int main (int argc, char *argv[])

  Main function for CLI to generate perlin noise.

7.12.1 Function Documentation

7.12.1.1 main()

```
int main (
            int argc,
            char * argv[])
```

Main function for CLI to generate perlin noise.

Parameters

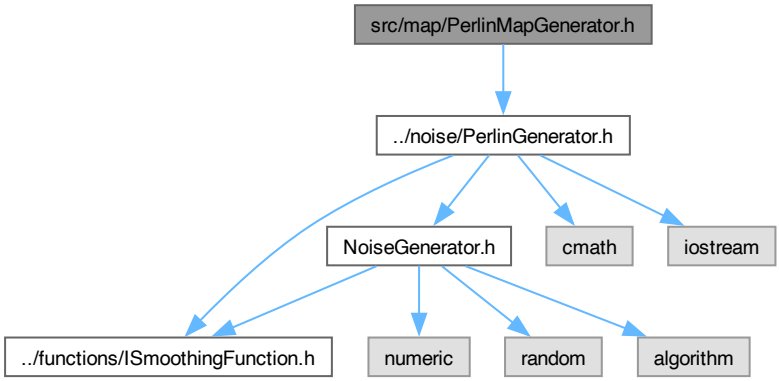
argc	the number of args passsed to the CLI
argv	the list of args passed to the CLI

Returns

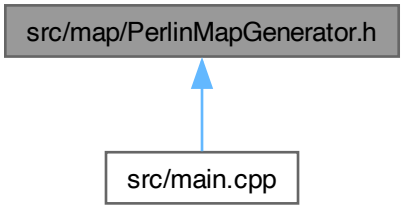
int the default UNIX like system return code

7.13 src/map/PerlinMapGenerator.h File Reference

`#include "../noise/PerlinGenerator.h"`
Include dependency graph for PerlinMapGenerator.h:



This graph shows which files directly or indirectly include this file:



Classes

- class PerlinMapGenerator

7.14 PerlinMapGenerator.h

Go to the documentation of this file.

```
#pragma once

#include "../noise/PerlinGenerator.h"

class PerlinMapGenerator
{

```

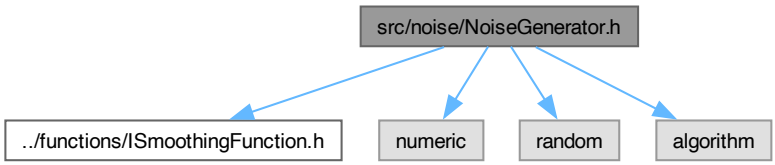
```
private:
    int octave;
    int width;
    int height;

    float persistence;
    float lacunarity;
    float scale;

    float base_frequency;
    float base_amplitude;

    std::vector<std::vector<float> perlin_values;

public:
    PerlinMapGenerator(
        int octave = 6,
        float persistence = 0.5f,
        float lacunarity = 2.0f,
        float scale = 4.5f,
        float base_frequency = 1.0f,
        float base_amplitude = 1.0f,
        int width = 1000,
        int height = 1000)
    {
        this->octave = octave;
        this->persistence = persistence;
        this->lacunarity = lacunarity;
        this->scale = scale;
        this->base_frequency = base_frequency;
        this->base_amplitude = base_amplitude;
        this->width = width;
        this->height = height;
        this->perlin_values = std::vector<std::vector<float>(
            width,
            std::vector<float>(height, 0.0f) // Fill with default value of 0.0f
        );
    }

    std::vector<std::vector<float> generate(ISmoothingFunction const &smoothing)
    {
        PerlinGenerator generator;
        for (int i = 0; i < octave; i++)
        {
            float frequency = this->base_frequency * std::pow(this->lacunarity, i);
            float amplitude = this-> base_amplitude * std::pow(this->persistence, i);

            for (int x = 0; x < this->width; ++x)
            {
                for (int y = 0; y < height; ++y)
                {
                    float sample_x = (static_cast<float>(x) / static_cast<float>(this->width)) * this->scale * frequency;
                    float sample_y = (static_cast<float>(y) / static_cast<float>(this->height)) * this->scale * frequency;
                    perlin_values[x][y] += generator.noise(sample_x, sample_y, smoothing) * amplitude;
                }
            }
        }

        float max_amplitude = 0.0f;
        for (int i = 0; i < octave; i++)
        {
            max_amplitude += base_amplitude * std::pow(persistence, i);
        }

        for (int x = 0; x < width; ++x)
        {
            for (int y = 0; y < height; ++y)
            {
                perlin_values[x][y] /= max_amplitude;
            }
        }
        return perlin_values;
    }
};
```

7.15 src/noise/NoiseGenerator.h File Reference

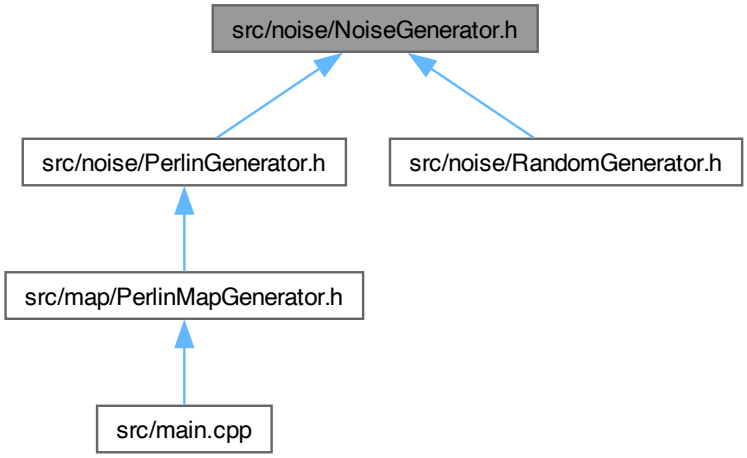
```
#include "../functions/ISmoothingFunction.h"
#include <numeric>
```

```
#include <random>
#include <algorithm>
```

Include dependency graph for NoiseGenerator.h:



This graph shows which files directly or indirectly include this file:



Classes

- class NoiseGenerator

  Partially abstract class providing tools to generate a noise.

7.16 NoiseGenerator.h

Go to the documentation of this file.

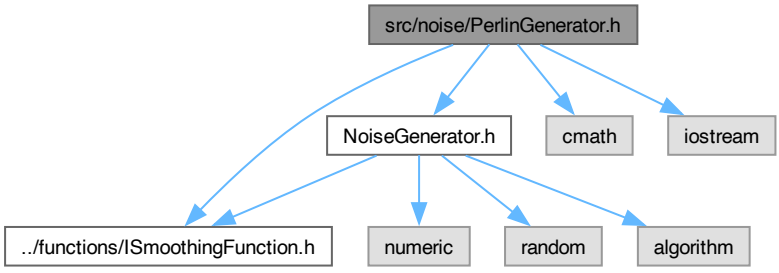
```
00001 #pragma once
00002
00003 #include "../functions/ISmoothingFunction.h"
00004
00005 #include <numeric>
00006 #include <random>
00007 #include <algorithm>
```

```
class NoiseGenerator {

public:

    virtual float noise(float x, float y, ISmoothingFunction const &smoothing_function) = 0;

protected:

    static int* getPermutationTable() {
        int* random_values = NoiseGenerator::getRandomValues();
        int* permutation_table = new int[512];

        for(int i = 0; i < 512; i++) {
            permutation_table[i] = random_values[i & 255];
        }

        delete[] random_values;
        return permutation_table;
    }

private:

    static int* getRandomValues() {
        int* values = new int[256];

        std::iota(values, values + 256, 1);

        std::random_device rd;
        std::mt19937 g(rd());

        std::shuffle(values, values + 256, g);

        return values;
    }
};
```

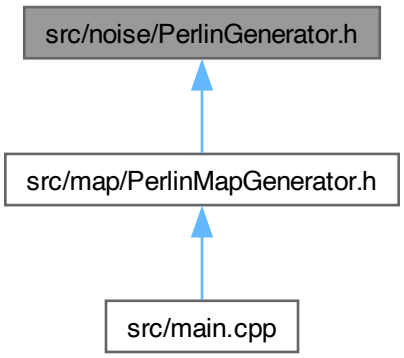
7.17 src/noise/PerlinGenerator.h File Reference

```
#include "NoiseGenerator.h"
#include "../functions/ISmoothingFunction.h"
#include <cmath>
#include <iostream>
```

Include dependency graph for PerlinGenerator.h:



This graph shows which files directly or indirectly include this file:



Classes

- class PerlinGenerator

  Implementation of a noise generator based on Perlin Algorithm.

7.18 PerlinGenerator.h

Go to the documentation of this file.

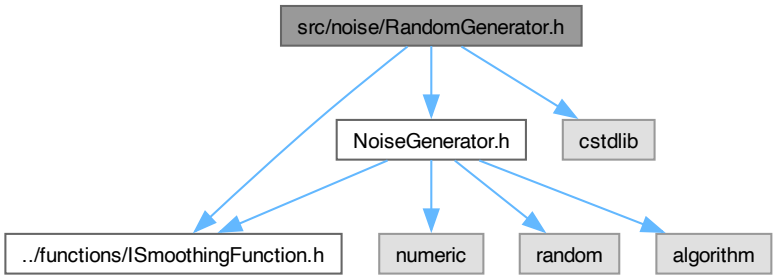
```
#pragma once

#include "NoiseGenerator.h"
#include "../functions/ISmoothingFunction.h"

#include <cmath>
#include <iostream>

class PerlinGenerator : public NoiseGenerator {

public:

    int* permutation_table;

    // The translation unit for the gradients, to ensure they are unit vectors.
    const float TRANSLATION_UNIT = 1.0f / std::sqrt(2.0f);

    // 8 possible gradients for 2D Perlin Noise
    const float GRADIENT[8][2] = {
        {TRANSLATION_UNIT,TRANSLATION_UNIT},
        {-TRANSLATION_UNIT,TRANSLATION_UNIT},
        {TRANSLATION_UNIT,-TRANSLATION_UNIT},
        {-TRANSLATION_UNIT,-TRANSLATION_UNIT},
        {1,0},
        {-1,0},
        {0,1},
        {0,-1}
    };

    PerlinGenerator() {
        this->permutation_table = this->getPermutationTable();
    }

    ~PerlinGenerator() {
        delete[] this->permutation_table;
    }

```

```
    float noise(float x, float y, ISmoothingFunction const &smoothing_function) override {

        int x0 = static_cast<int>(std::floor(x)) & 255;
        int y0 = static_cast<int>(std::floor(y)) & 255;

        int gradient_index_00 = this->permutation_table[this->permutation_table[x0] + y0] % 8;
        int gradient_index_10 = this->permutation_table[this->permutation_table[x0 + 1] + y0] % 8;
        int gradient_index_01 = this->permutation_table[this->permutation_table[x0] + y0 + 1] % 8;
        int gradient_index_11 = this->permutation_table[this->permutation_table[x0 + 1] + y0 + 1] % 8;

        float dot_00 = this->GRADIENT[gradient_index_00][0] * (x - std::floor(x)) + this->GRADIENT[gradient_index_00][1] * (y - std::floor(y));
        float dot_10 = this->GRADIENT[gradient_index_10][0] * (x - std::floor(x) - 1) + this->GRADIENT[gradient_index_10][1] * (y - std::floor(y));
        float dot_01 = this->GRADIENT[gradient_index_01][0] * (x - std::floor(x)) + this->GRADIENT[gradient_index_01][1] * (y - std::floor(y) - 1);
        float dot_11 = this->GRADIENT[gradient_index_11][0] * (x - std::floor(x) - 1) + this->GRADIENT[gradient_index_11][1] * (y - std::floor(y) - 1);

        float u = smoothing_function.apply(x - std::floor(x));
        float v = smoothing_function.apply(y - std::floor(y));
        float lerp_0 = (1 - u) * dot_00 + u * dot_10;
        float lerp_1 = (1 - u) * dot_01 + u * dot_11;
        return (1 - v) * lerp_0 + v * lerp_1;
    }
};
```

7.19 src/noise/RandomGenerator.h File Reference

```
#include "../functions/ISmoothingFunction.h"
#include "NoiseGenerator.h"
#include <cstdlib>
```

Include dependency graph for RandomGenerator.h:



Classes

- class RandomGenerator

7.20 RandomGenerator.h

Go to the documentation of this file.

```
#pragma once

#include "../functions/ISmoothingFunction.h"
#include "NoiseGenerator.h"

```

```
#include <cstdlib>

class RandomGenerator : public NoiseGenerator {

public:

    float noise(float x, float y, ISmoothingFunction const &smoothing_function) override {
        return smoothing_function.apply((rand()) / static_cast <float> (RAND_MAX));
    }
};
```

Index

~ISmoothingFunction
    ISmoothingFunction, 18
~PerlinGenerator
    PerlinGenerator, 22

apply
    BasicSmooth, 14
    ISmoothingFunction, 18

b
    BiomeColor, 15
base_amplitude
    PerlinMapGenerator, 24
base_frequency
    PerlinMapGenerator, 24
BasicSmooth, 13
    apply, 14
Biome
    Biome.h, 30
Biome.h
    Biome, 30
    DeepWater, 30
    Forest, 30
    get_biome, 30
    get_biome_color, 30
    Hills, 30
    Mountain, 30
    Plains, 30
    Reef, 30
    ShallowWater, 30
    Shore, 30
    Snow, 30
BiomeColor, 15
    b, 15
    g, 15
    r, 15

DeepWater
    Biome.h, 30

Forest
    Biome.h, 30

g
    BiomeColor, 15
generate
    PerlinMapGenerator, 24
generate_png_image
    ImageGenerator, 16
get_biome
    Biome.h, 30
get_biome_color
    Biome.h, 30
getPermutationTable
    NoiseGenerator, 19
getRandomValues
    NoiseGenerator, 19
GRADIENT
    PerlinGenerator, 22

height
    PerlinMapGenerator, 24
Hills
    Biome.h, 30

ImageGenerator, 15
    generate_png_image, 16
    set_pixel, 16
ISmoothingFunction, 17
    ~ISmoothingFunction, 18
    apply, 18

lacunarity
    PerlinMapGenerator, 24

main
    main.cpp, 61
main.cpp
    main, 61
Mountain
    Biome.h, 30

noise
    NoiseGenerator, 19
    PerlinGenerator, 22
    RandomGenerator, 26
NoiseGenerator, 18
    getPermutationTable, 19
    getRandomValues, 19
    noise, 19

octave
    PerlinMapGenerator, 24

perlin_values
    PerlinMapGenerator, 24
PerlinGenerator, 20
    ~PerlinGenerator, 22
    GRADIENT, 22
    noise, 22
    PerlinGenerator, 22
    permutation_table, 22

TRANSLATION_UNIT, 23
PerlinMapGenerator, 23
base_amplitude, 24
base_frequency, 24
generate, 24
height, 24
lacunarity, 24
octave, 24
perlin_values, 24
PerlinMapGenerator, 23
persistence, 24
scale, 24
width, 24
permutation_table
PerlinGenerator, 22
persistence
PerlinMapGenerator, 24
Plains
Biome.h, 30

r
BiomeColor, 15
RandomGenerator, 25
noise, 26
Reef
Biome.h, 30

scale
PerlinMapGenerator, 24
set_pixel
ImageGenerator, 16
ShallowWater
Biome.h, 30
Shore
Biome.h, 30
Snow
Biome.h, 30
src Directory Reference, 12
src/functions Directory Reference, 9
src/functions/BasicSmooth.h, 27, 28
src/functions/ISmoothingFunction.h, 28, 29
src/image Directory Reference, 10
src/image/Biome.h, 29, 31
src/image/ImageGenerator.h, 32, 33
src/image/stb_image_write.cpp, 34
src/libs Directory Reference, 10
src/libs/stb_image_write.h, 35, 39
src/main.cpp, 60
src/map Directory Reference, 11
src/map/PerlinMapGenerator.h, 62
src/noise Directory Reference, 11
src/noise/NoiseGenerator.h, 63, 64
src/noise/PerlinGenerator.h, 65, 66
src/noise/RandomGenerator.h, 67
stb_image_write.cpp
STB_IMAGE_WRITE_IMPLEMENTATION, 34
stb_image_write.h
stbi_flip_vertically_on_write, 36
stbi_write_bmp, 36
stbi_write_bmp_to_func, 37
stbi_write_force_png_filter, 39
stbi_write_func, 36
stbi_write_hdr, 37
stbi_write_hdr_to_func, 37
stbi_write_jpg, 37
stbi_write_jpg_to_func, 37
stbi_write_png, 38
stbi_write_png_compression_level, 39
stbi_write_png_to_func, 38
stbi_write_tga, 38
stbi_write_tga_to_func, 38
stbi_write_tga_with_rle, 39
STBIWDEF, 36
STB_IMAGE_WRITE_IMPLEMENTATION
stb_image_write.cpp, 34
stbi_flip_vertically_on_write
stb_image_write.h, 36
stbi_write_bmp
stb_image_write.h, 36
stbi_write_bmp_to_func
stb_image_write.h, 37
stbi_write_force_png_filter
stb_image_write.h, 39
stbi_write_func
stb_image_write.h, 36
stbi_write_hdr
stb_image_write.h, 37
stbi_write_hdr_to_func
stb_image_write.h, 37
stbi_write_jpg
stb_image_write.h, 37
stbi_write_jpg_to_func
stb_image_write.h, 37
stbi_write_png
stb_image_write.h, 38
stbi_write_png_compression_level
stb_image_write.h, 39
stbi_write_png_to_func
stb_image_write.h, 38
stbi_write_tga
stb_image_write.h, 38
stbi_write_tga_to_func
stb_image_write.h, 38
stbi_write_tga_with_rle
stb_image_write.h, 39
STBIWDEF
stb_image_write.h, 36

TRANSLATION_UNIT
PerlinGenerator, 23

width
PerlinMapGenerator, 24